MODUL PRAKTIKUM

CE649 PENGOLAHAN CITRA
PROGRAM SARJANA S1TEKNIK KOMPUTER
FAKULTAS TEKNIK DAN INFORMATIKA





PENYUSUN : DR. NABILA HUSNA SHABRINA, S.T., M.T.

PROGRAM STUDI TEKNIK KOMPUTER
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA
Gedung B Lantai 5, Kampus UMN
Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia
Telp: +62-21.5422.0808 (ext. 1803), email: ict.lab@umn.ac.id, web: umn.ac.id

DAFTAR ISI

DAFTAR ISI	2
CAPAIAN PEMBELAJARAN PRAKTIKUM	3
MODUL 1 PENGENALAN PYTHON	4
MODUL 2 MANIPULASI DAN TRANSFORMASI GAMBAR DENGAN PYTHON	9
MODUL 3 FILTER GAMBAR	20
MODUL 4 MORPHOLOGY IMAGE PROCESSING	26
MODUL 5 IMAGE SEGMENTATION	32
MODUL 6 FEATURE DETECTION & DESCRIPTION	44
MODUL 7 SUPERVISED LEARNING UNTUK ANALISIS GAMBAR	53
MODUL 8 UNSUPERVISED LEARNING UNTUK ANALISIS GAMBAR	64
MODUL 9 CNN UNTUK KLASIFIKASI GAMBAR (1)	75
MODUL 10 CNN UNTUK KLASIFIKASI GAMBAR (2)	78

Level KKNI: 6

D	Mampu bertanggungjawab atas pekerjaan yang dilakukan, sebagai individu maupun sebagai pemimpin dalam kelompok.
F	Mampu mengidentifikasi masalah berdasarkan hasil analisis data, serta memberikan solusi yang tepat dengan dasar pemikiran logis, kritis, sistematis, dan inovatif.
I	Mampu melakukan rancang bangun dan integrasi perangkat lunak dengan perangkat keras, sebagai suatu sistem terdistribusi yang terhubung dalam sebuah infrastruktur, guna memecahkan berbagai permasalahan rekayasa dengan adaptif.

MODUL 1
PENGENALAN PYTHON

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari konsep dasar fungsi-fungsi image processing pada Python
Sub pokok bahasan yang akan dipelajari pada pertemuan ini yaitu
1. Install Python
2. Fungsi-fungsi dasar Python

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menjelaskan dasar-dasar bahasa pemrograman Python (C_3)

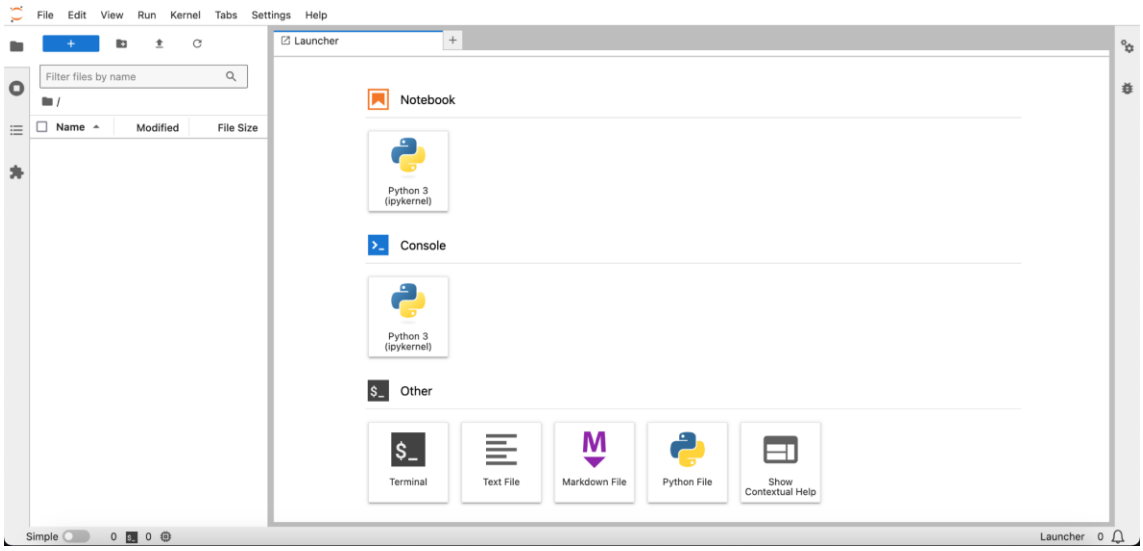
PENUNJANG PRAKTIKUM

1. Koneksi internet
2. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

- **Install Python**

1. Download Python versi 3.12.5 pada web https://www.python.org/downloads/
2. Install Jupyter Lab dengan mengetikkan 'pip install jupyterlab' pada terminal
3. Setelah instalasi berhasil, Jupyter Lab dapat dijalankan dengan mengetikkan 'jupyter lab' pada terminal



Gambar 1. Tampilan awal Jupyter Lab

4. Create notebook baru, lalu lakukan beberapa perhitungan sederhana seperti menampilkan penjumlahan, perkalian, dan pembagian

Catatan : Implementasi program juga bisa dilakukan menggunakan google colab (https://colab.research.google.com/)

5. Contoh perhitungan sederhana dapat dilihat pada code di bawah

```
a = 3
b = 2

print(a+b)
print(a-b)
print(a*b)
print(a/b)
```

```
5
1
6
1.5
```

Gambar 2. Contoh output perhitungan sederhana

- **Analisis Gambar dengan Python**

6. Siapkan gambar yang akan dianalisis
7. Install library yang dibutuhkan dengan mengetikkan 'pip install pillow scikit-image matplotlib scipy opencv-python' pada terminal
8. Masukkan code berikut untuk melakukan import beberapa *libraries*

```
from PIL import Image
from skimage.io import imread, imshow, show
import matplotlib.pylab as plt
import matplotlib.image as mpimg
import imageio.v2 as imageio
import numpy
import cv2
```

Read image with Pillow

9. Baca gambar yang telah disiapkan dengan code im= Image.open("Directory/namafile")
10. Setelah gambar terbaca, tampilkan gambar dengan code Im.show() , beberapa informasi gambar juga bisa ditampilkan dengan code print(im.width, im.height, im.mode, im.format)

```
im = Image.open("sample.png")
im.show()
print(im.width, im.height, im.mode, im.format)
```

```
1000 400 P PNG
```

Read image with scipy

11. Baca gambar yang telah disiapkan dengan code im2= imageio.imread("Directory/namafile")
12. Setelah gambar terbaca, tampilkan gambar dengan code plt.imshow(im2), ukuran gambar bisa ditampilkan dengan code print(im2.shape)

```
im2 = imageio.imread("sample.png")
plt.imshow(im2)
print(im2.shape)
```

```
(400, 1000, 3)
```



Read image with Matplotlib

13.
```
import matplotlib.image as mpimg
```
Baca gambar yang telah disiapkan dengan code `im3 = mpimg.imread("Directory/namafile")`
14. Setelah gambar terbaca, tampilkan gambar dengan code `plt.imshow(im3)`, ukuran gambar bisa ditampilkan dengan code `print(im3.shape)`
15. Tampilan gambar pada layar bisa diperbesar/diperkecil ukurannya dengan menambahkan code tampilkan gambar dengan code `plt.figure(figsize=(x,x))`, axis pada gambar bisa dihilangkan dengan menambahkan code `plt.axis('off')`

```
im3 = mpimg.imread("sample.png")
plt.figure(figsize=(5,5))
plt.axis('off')
plt.imshow(im3)
print(im3.shape)
```

```
(400, 1000, 4)
```



Resize image

16. Baca gambar yang akan diresize
17. Untuk memperbesar ukuran gambar tambahkan code `imu1 = im.resize((im.width*x, im.height*y))` dengan x dan y adalah faktor pengali ukuran gambar, lalu tampilkan gambar dengan code `plt.imshow(imu)`

```
x = 2
y = 2
imu1 = im.resize((im.width*x, im.height*y))
plt.imshow(imu1)
print(imu1.width, imu1.height, imu1.mode, imu1.format)
```

```
2000 800 P None
```



18. Untuk memperkecil ukuran gambar tambahkan code `imu2 = im.resize((im.width//x, im.height//y))` dengan x dan y adalah faktor pembagi ukuran gambar, lalu tampilkan gambar dengan code `plt.imshow(imu)`

```
imu2 = im.resize((im.width//x, im.height//y))
plt.imshow(imu2)
print(imu2.width, imu2.height, imu2.mode, imu2.format)
```

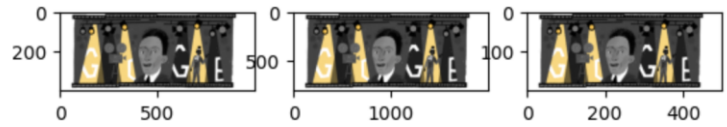
```
500 200 P None
```



19. Untuk menampilkan beberapa gambar sekaligus dapat digunakan code

```
fig, ax = plt.subplots(1,3)
ax[0].imshow(im);
ax[1].imshow(imu1);
ax[2].imshow(imu2);
```

```
fig, ax = plt.subplots(1,3)
ax[0].imshow(im);
ax[1].imshow(imu1);
ax[2].imshow(imu2);
```



Read image with OpenCV

20. Baca gambar yang telah disiapkan dengan code `im= cv2.imread("directory/namafile")`

21. Untuk menampilkan gambar tulis code berikut

```
cv2.imshow("image",im)##display
cv2.waitKey(0) ##wait indefinately for a key stroke
cv2.destroyAllWindows()##destroys all windows
```

22. Untuk mengubah gambar dengan warna *grayscale*, baca gambar dengan code `cv2.imread("directory/namafile", 0)` lalu tampilkan gambar dengan code yang sama dengan step sebelumnya

REFERENSI

1. Sandipan Dey, Hands on Image Processing with Phyton, Packt Publishing, 2018 [SD]
2. Opencv.org

MODUL 2
MANIPULASI DAN TRANSFORMASI GAMBAR DENGAN PYTHON

DESKRIPSI TEMA

Mahasiswa mampu menggunakan metode-metode manipulasi dan transformasi gambar dengan Python(C3)

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan metode-metode manipulasi dan transformasi gambar dengan Python(C3)

PENUNJANG PRAKTIKUM

1. Koneksi internet
2. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

1. Import beberapa libraries

```
# import some libraries
from PIL import Image, ImageFont, ImageDraw
from PIL.ImageChops import add, subtract
from skimage.io import imread, imshow, show
from skimage.transform import SimilarityTransform, warp, swirl
from scipy.ndimage import affine_transform
import matplotlib.image as mpimg
import matplotlib.pylab as plt
```

Penjelasan fungsi masing-masing library:

- PIL atau Pillow berfungsi untuk membaca dan menulis gambar dalam berbagai format, dan juga menyediakan operasi manipulasi gambar dasar, seperti cropping, rotation, draw, dsb.
- skimage merupakan library hasil pengembangan dari Numpy dan scikit-learn yang juga dapat membaca dan menulis gambar dalam berbagai format, dan menyediakan sejumlah fungsi manipulasi maupun transformasi gambar
- scipy adalah sebuah library yang umumnya digunakan untuk scientific dan technical computing. Untuk image processing, tersedia juga beberapa fungsi untuk image filtering, morphological operations, geometric transformation, dan juga edge detection.
- matplotlib merupakan library yang umumnya digunakan untuk melakukan plotting data maupun gambar. Library ini juga menyediakan fungsi untuk melakukan blending pada gambar.

Blending two images using cross-dissolving

2. Siapkan **2 buah gambar yang berbeda dengan ukuran pixel yang sama**.
3. Dalam contoh ini digunakan gambar flower1.jpg dan flower2.jpg seperti yang ditunjukkan pada tampilan berikut



Flower 1.jpg



Flower2.jpg

4. Untuk melakukan blending 2 buah gambar menggunakan cross dissolving, gunakan syntax di bawah.

```
im1 = mpimg.imread("flower1.jpg") / 255 # scale RGB values in [0,1]
im2 = mpimg.imread("flower2.jpg") / 255
i = 1
plt.figure(figsize=(18,15))
for alpha in np.linspace(0,1,20):
    plt.subplot(4,5,i)
    plt.imshow((1-alpha)*im1 + alpha*im2)
    plt.axis('off')
    i += 1
plt.subplots_adjust(wspace=0.05, hspace=0.05)
plt.show()
```

Untuk pemanggilan gambar, syntax `im1= mpig.imread("flower1.jpg")` **disesuaikan dengan directory dan nama file gambar yang sudah disiapkan sebelumnya**

Gambar hasil dari metode blending 2 buah gambar menggunakan cross dissolving ditunjukkan seperti pada Gambar 1.



Gambar 1 Blending two images using cross-dissolving

Image manipulation with PIL

5. Buka dan tampilkan informasi gambar menggunakan syntax library PIL

```
im3 = Image.open("flower.jpg")
print (im3.width, im3.height, im3.mode, im3.format)
```

Nama gambar disesuaikan dengan directory dan nama file gambar yang sudah disiapkan sebelumnya

Cropping an Image

6. Cropping gambar dilakukan menggunakan syntax `crop()`. Secara lengkap, cropping gambar dapat dilakukan dengan menggunakan syntax berikut

```
#crop the rectangle given by (left,top, right, bottom)
im3_crop = im3.crop((175,75,320,200))
im3_crop.show()
```

Sesuaikan ukuran cropping pada syntax di atas sesuai dengan kebutuhan.

Negating an Image

7. Negating image dapat dilakukan menggunakan syntax `point()`. Secara lengkap, negating sebuah gambar dapat dilakukan dengan menggunakan syntax berikut

```
im3_negative = im3.point(lambda x:255-x)
im3_negative.show()
```

Ukuran pixel pada gambar direpresentasikan menggunakan 1-byte unsigned integers sehingga jika pixel tersebut dikurangkan terhadap nilai maksimumnya maka akan diperoleh gambar "negative" dari gambar sebelumnya. Hasil syntax di atas ditunjukkan pada Gambar 2.



Gambar 2. Hasil negating image

Converting an image into grayscale

8. Ubah gambar menjadi grayscale menggunakan syntax `convert()` dengan parameter 'L' seperti syntax berikut.

```
im_grayscale = im3.convert('L')
im_grayscale.show()
```

Hasil dari syntax tersebut ditunjukkan pada Gambar 3.



Gambar 3. Hasil grayscale image

Some gray-level transformations

Dengan menggunakan syntax `point()`, masing-masing nilai pixel dari gambar input dapat ditransfer ke nilai pixel yang sesuai pada gambar output. Ada dua transformasi gray-level yang akan digunakan dalam praktikum ini, yaitu :

Log transformation dapat digunakan untuk mengompres gambar dengan rentang nilai pixel yang dinamis

Power law transformation dapat digunakan untuk mengoreksi nilai γ

9. Ubah gambar menggunakan log transformation dengan syntax berikut.

```
im_grayscale.point(lambda x: 255*np.log(1+x/255)).show()
```

Hasil dari syntax tersebut ditunjukkan pada Gambar 4.



Gambar 4. Hasil log transformation

10. Ubah gambar menggunakan power-law transformation dengan syntax berikut. Gunakan nilai $\gamma = 0.6$

```
im_grayscale.point(lambda x: 255*(x/255)**0.6).show()
```

Hasil dari syntax tersebut ditunjukkan pada Gambar 5.



Gambar 5. Hasil power-law transformation

Reflecting an Image

11. Untuk mencerminkan gambar, dapat digunakan syntax $\mathrm{transpose}()$. Secara lengkap, pencerminan gambar dapat dilakukan dengan menggunakan syntax berikut

```
# reflect about the vertical axis
im3_rotate1 = im3.transpose(Image.FLIP_LEFT_RIGHT).show()
#reflect about the horizontal axis
im3_rotate2 = im3. transpose(Image.FLIP_TOP_BOTTOM).show()
```

Hasil syntax di atas ditunjukkan pada Gambar 6.



Gambar 6. Hasil rotate vertical axis (atas) dan rotate horizontal axis (bawah)

Rotating an image

12. Untuk memutar gambar dapat digunakan syntax `rotate()` yang secara lengkap dapat ditulis dengan syntax berikut. Gunakan putaran sudut sebesar 45°

```
im3_rotate45= im3.rotate(45).show()
```

Hasil syntax di atas ditunjukkan pada Gambar 7.



Gambar 7. Hasil rotate 45 derajat

Adding salt and pepper noise to an image

13. Untuk menambahkan noise, dapat digunakan syntax putpixel(). putpixel() secara umum digunakan untuk mengubah nilai pixel pada suatu gambar. Secara lengkap, syntax dapat dituliskan sebagai berikut.

```
# choose 5000 random locations inside image
imcopy = im3.copy() # keep the original image, create a copy\
n = 5000
x, y = np.random.randint(0, imcopy.width, n), np.random.randint(0, imcopy.height,n)
for (x,y) in zip(x,y):
    imcopy.putpixel((x, y), ((0,0,0) if np.random.rand() < 0.5 else
    (255,255,255)))
imcopy.show()
```

Hasil syntax di atas ditunjukkan pada Gambar 8.



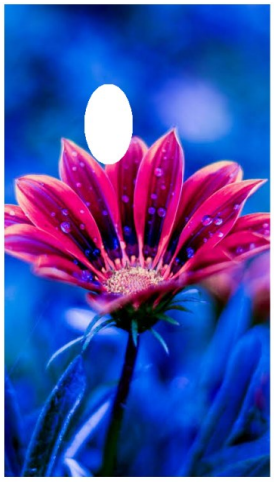
Gambar 8. Hasil adding salt & pepper noise

Drawing on an image

14. Untuk menambahkan sebuah objek pada gambar, dapat digunakan module PIL.ImageDraw. Objek berbentuk elips dapat ditambahkan dengan syntax ellipse(). Secara lengkap, penambahan objek elips dapat dilakukan dengan menggunakan syntax berikut

```
im4 = Image.open('flower3.jpg')
draw = ImageDraw.Draw(im4)
draw.ellipse((125, 125, 200, 250), fill= (255,255,255,128))
del draw
im4.show()
```

Hasil syntax di atas ditunjukkan pada Gambar 9.



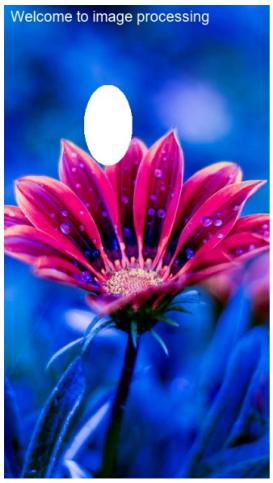
Gambar 9. Hasil penambahan objek elips pada gambar

Drawing text on an image

15. Teks dapat ditambahkan pada sebuah gambar dengan menggunakan syntax `text()`. Secara lengkap, penambahan teks dapat dilakukan dengan menggunakan syntax berikut

```
draw = ImageDraw.Draw(im4)
font = ImageFont.truetype("arial.ttf", 23) # use a truetype font
draw.text((10, 5), "Welcome to image processing", font=font)
del draw
im4.show()
```

Hasil syntax di atas ditunjukkan pada Gambar 10.



Gambar 10. Hasil penambahan teks pada gambar

Superimposing two images

16. Penempatan (superimposing) sebuah gambar di atas gambar lainnya dapat dilakukan menggunakan perkalian dua buah gambar. Secara lengkap, proses penempatan suatu gambar di atas gambar lainnya dapat dilakukan dengan menggunakan syntax berikut

```
im5 = Image.open("flower4.jpg")
im6 = Image.open("flower5.jpg").convert('RGB').resize((im5.width,im5.height))
multiply(im5, im6).show()
```

Hasil sebelum dan sesudah syntax di atas ditunjukkan pada Gambar 11.



Gambar 11. Gambar flower 4 (atas), flower 5(tengah), hasil superimposing gambar (bawah)

Adding two images

17. Untuk menambahkan dua buah gambar dapat dilakukan dengan syntax berikut.

```
add(im5,im6).show()
```

Hasil syntax di atas ditunjukkan pada Gambar 12.



Gambar 12. Hasil adding gambar

Subtracting two images

18. Untuk mengurangkan dua buah gambar dapat dilakukan dengan syntax berikut.

```
subtract(im5,im6).show()
```

Hasil syntax di atas ditunjukkan pada Gambar 13.



Gambar 13. Hasil subtracting gambar

Image manipulation with scikit-image

Applying an Affine transformation on an image

19. Affine transformation dapat dilakukan menggunakan syntax SimilarityTransform() dan warp(). Secara lengkap affine transformation dapat dilakukan dengan syntax berikut.

```
im = imread("parrot.jpg")
tform = SimilarityTransform(scale=0.9, rotation=np.pi/4,translation=(im.shape[0]/2, -100))
warped = warp(im, tform)
plt.imshow(warped), plt.axis('off'), plt.show()
```

Hasil syntax di atas ditunjukkan pada Gambar 14.



```
(<matplotlib.image.AxesImage at 0x1e60853a448>,
 (-0.5, 763.5, 400.5, -0.5),
 None)
```

Gambar 14. Hasil affine transformation

Applying the swirl transform

20. Swirl transform dapat dilakukan menggunakan syntax berikut

```
im = imread("parrot.jpg")
swirled = swirl(im, rotation=0,
strength=15, radius=200)
plt.imshow(swirled), plt.axis('off'), plt.show()
```

Hasil syntax di atas ditunjukkan pada Gambar 15.



```
(<matplotlib.image.AxesImage at 0x1e6091a9848>,
 (-0.5, 763.5, 400.5, -0.5),
 None)
```

Gambar 15. Hasil swirl transform

REFERENSI

1. Sandipan Dey, Hands on Image Processing with Phyton, Packt Publishing, 2018, **Chapter 2**

MODUL 3
FILTER GAMBAR

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari beberapa metode dasar pemfilteran gambar

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan metode-metode pemfilteran gambar dengan Python (C_3)

PENUNJANG PRAKTIKUM

1. Koneksi internet
2. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Calculate & Plot Histogram of an Image

1. Import beberapa libraries

```
import matplotlib.pylab as plt
import numpy as np
import cv2
```

2. Siapkan **gambar.**
3. Untuk melakukan perhitungan histogtam, gunakan salah satu syntax di bawah.

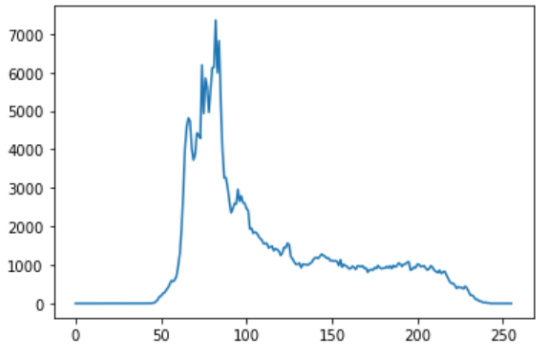
```
im1 = cv2.imread('flower1.jpg',0)
hist = cv2.calcHist([im1],[0],None,[256],[0,256])
```

```
hist,bins = np.histogram(im1.ravel(),256,[0,256])
```

Untuk pemanggilan gambar, syntax $\mathrm{im1} = \mathrm{cv2.imread('flower1.jpg',0)}$ **disesuaikan dengan directory dan nama file gambar yang sudah disiapkan sebelumnya**

4. Plot menggunakan syntax plt.plot(hist)
   Contoh hasil :

```
Out[3]: [<matplotlib.lines.Line2D at 0x217a7f279d0>]
```



5. Untuk plot hasil histogram dengan 3 channel warna yang berbeda (r,g, dan b), gunakan syntax di bawah

```
im1 = cv2.imread('flower1.jpg')
color = ('b','g','r')
for i,col in enumerate(color):
    histr = cv2.calcHist([im1],[i],None,[256],[0,256])
    plt.plot(histr,color = col)
    plt.xlim([0,256])
plt.show()
```

Contoh hasil :

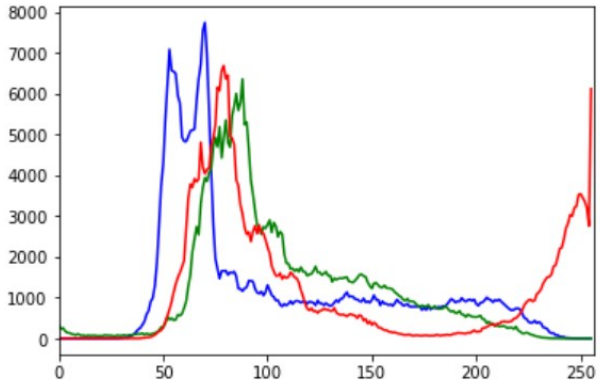


Image smoothing/Image blurring

6. Siapkan **gambar.**
7. Untuk melakukan image smoothing dengan **kernel 5x5,** gunakan syntax berikut.

```
im3 = cv2.imread('images/merak.jpg')

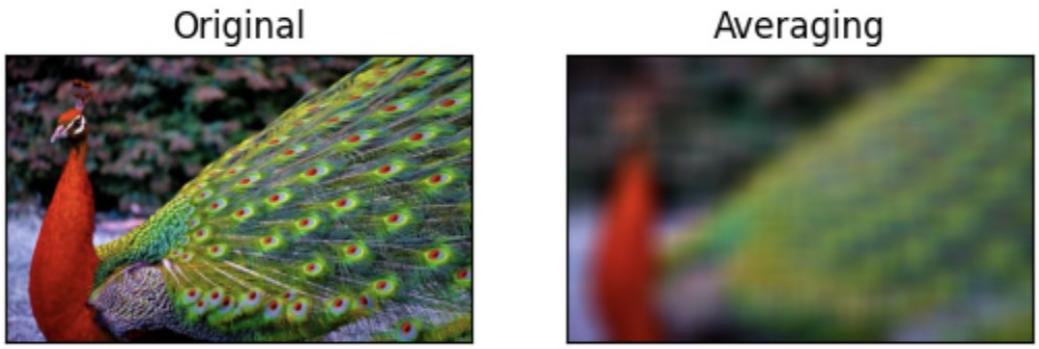
kernel = np.ones((55,55),np.float32)/3025
dst = cv2.filter2D(im3,-1,kernel)

plt.subplot(121),plt.imshow(im3),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(dst),plt.title('Averaging')
plt.xticks([]), plt.yticks([])
plt.show()
```

.

Nama gambar disesuaikan dengan directory dan nama file gambar yang sudah disiapkan sebelumnya

Contoh hasil :



8. Lakukan perubahan terhadap ukuran kernel, analisis hasilnya!

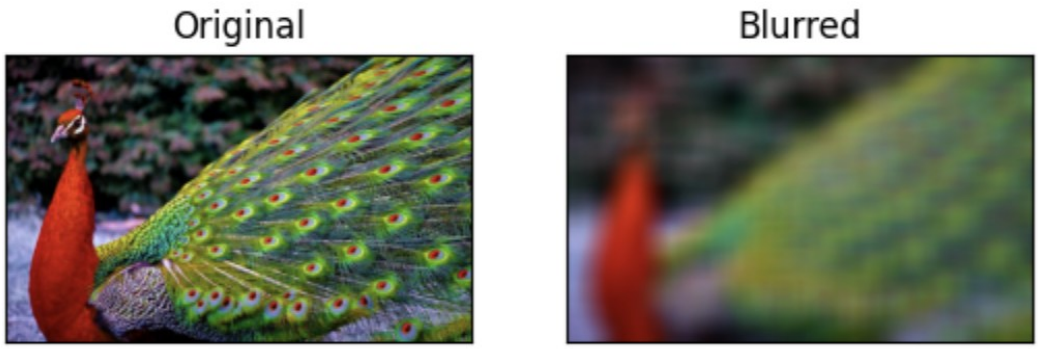
9. Untuk melakukan image blurring dengan metode **averaging,** gunakan syntax berikut.

```
im3 = cv2.imread('images/merak.jpg')

blur = cv2.blur(im3, (55,55))

plt.subplot(121),plt.imshow(im3),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(blur),plt.title('Blurred')
plt.xticks([]), plt.yticks([])
plt.show()
```

Contoh hasil :



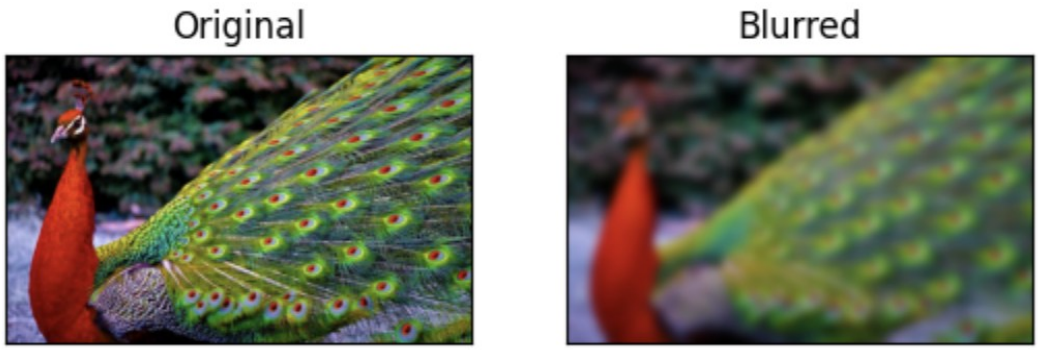
10. Untuk melakukan image smoothing dengan metode **gaussian filtering,** gunakan syntax berikut.

```
im3 = cv2.imread('images/merak.jpg')

blur = cv2.GaussianBlur(im3, (55,55), 0)

plt.subplot(121),plt.imshow(im3),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(blur),plt.title('Blurred')
plt.xticks([]), plt.yticks([])
plt.show()
```

Contoh hasil :



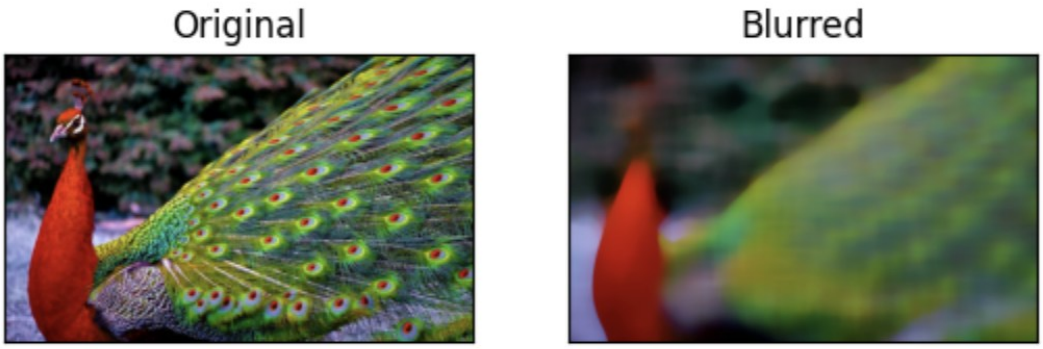
11. Untuk melakukan image smoothing dengan metode **median filtering,** gunakan syntax berikut.

```
im3 = cv2.imread('images/merak.jpg')

blur = cv2.medianBlur(im3, 55)

plt.subplot(121),plt.imshow(im3),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(blur),plt.title('Blurred')
plt.xticks([]), plt.yticks([])
plt.show()
```

Contoh hasil :



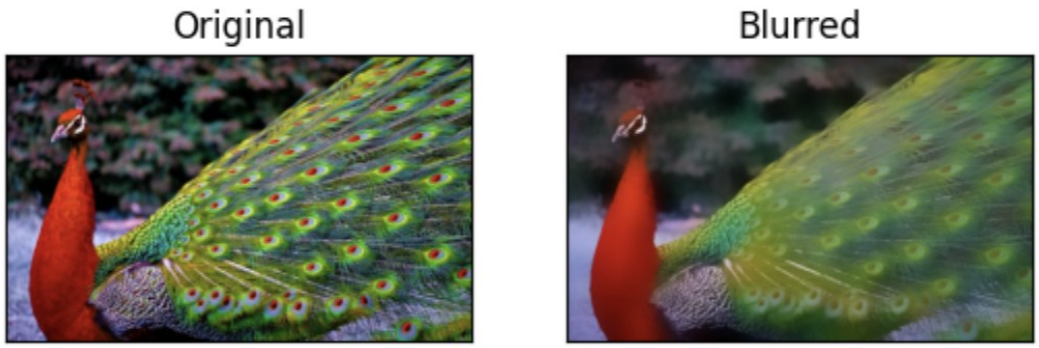
12. Untuk melakukan image smoothing dengan metode **bilateral filtering**, gunakan syntax berikut.

```
im3 = cv2.imread('images/merak.jpg')

blur = cv2.bilateralFilter(im3,55,155,155)

plt.subplot(121),plt.imshow(im3),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(blur),plt.title('Blurred')
plt.xticks([]), plt.yticks([])
plt.show()
```

Contoh hasil :



13. Bandingkan hasil image blurring menggunakan metode box filter dengan image blurring menggunakan metode gaussian filtering.

```
from scipy import misc, ndimage
from matplotlib.pyplot import imread

im4 = imread('images/merak.jpg')
k = 49 #7x7 kernel
im_box = ndimage.uniform_filter(im4, size=(k,k,1))
s = 7 #sigma value
t = (((k-1)/2)-0.5)/s #truncate parameter value for a kxk gaussian kernel with sigma s
im_gaussian = ndimage.gaussian_filter(im4, sigma=(s,s,0), truncate=t)
fig = plt.figure(figsize=(30,10))
plt.subplot(131),plt.imshow(im4),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(132),plt.imshow(im_box),plt.title('with the box filter')
plt.xticks([]), plt.yticks([])
plt.subplot(133),plt.imshow(im_gaussian),plt.title('with the gaussian filter')
plt.xticks([]), plt.yticks([])
plt.show()
```

Contoh hasil :



Image sharpened

14. Untuk melakukan image sharpened dengan metode metode laplacian, gunakan syntax berikut.

```
from skimage.filters import laplace
from skimage.color import rgb2gray

im5 = rgb2gray(imread('images/merak.jpg'))
im6 = np.clip(laplace(im5) + im5, 0, 1)
plt.figure(figsize=(20,30))
plt.subplot(121),plt.imshow(im5, cmap="gray"),plt.title('Original')
plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(im6, cmap="gray"),plt.title('Sharpened Image')
plt.xticks([]), plt.yticks([])
plt.tight_layout()
plt.show()
```

Contoh hasil :



REFERENSI

1. Sandipan Dey, Hands on Image Processing with Phyton, Packt Publishing, 2018, **Chapter 3-4**
2. https://opencv-python-tutroals.readthedocs.io

MODUL 4
MORPHOLOGY IMAGE PROCESSING

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari beberapa operator-operator morfologi dalam pengolahan gambar

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan operator-operator morfologi dalam pengolahan gambar dengan Python (C_3)

PENUNJANG PRAKTIKUM

1. Jupyter Lab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Morphology image processing menggunakan OpenCV

1. Import beberapa libraries

```
import cv2
import numpy as np
```

2. Siapkan **gambar dengan format binary.** JIka gambar belum dalam format binary, ubah menggunakan syntax berikut.

```
im_gray = cv2.imread('flower.jpg', cv2.IMREAD_GRAYSCALE)
(thresh, im_bw) = cv2.threshold(im_gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
thresh = 127
im_bw = cv2.threshold(im_gray, thresh, 255, cv2.THRESH_BINARY)[1]
thresh = 127
cv2.imwrite('bw_flower.jpg', im_bw)
```

Untuk pemanggilan gambar, syntax $\mathrm{im1} = \mathrm{cv2.imread}(\text{'flower1.jpg'})$ **disesuaikan dengan directory dan nama file gambar yang sudah disiapkan sebelumnya**

3. Hasil syntax tersebut ditunjukkan pada Gambar 1.




Gambar 1 Hasil binary image

4. Untuk melakukan operasi **erosion,** gunakan syntax berikut.

```
im = cv2.imread('bw_flower.jpg')
kernel = np.ones((5,5),np.uint8)
erosion = cv2.erode(im,kernel,iterations = 1)
cv2.imshow("image", erosion)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Hasil syntax tersebut ditunjukkan pada Gambar 2.




Gambar 2. Hasil operasi erosion

5. Untuk melakukan operasi **dilation,** gunakan syntax berikut.

```
dilation = cv2.dilate(im,kernel,iterations = 1)
```

6. Hasil syntax tersebut ditunjukkan pada Gambar 3.




Gambar 3. Hasil operasi dilation

7. Untuk melakukan operasi **opening,** gunakan syntax berikut.

```
opening = cv2.morphologyEx(im, cv2.MORPH_OPEN, kernel)
```

Hasil syntax tersebut ditunjukkan pada Gambar 4.



Gambar 4. Hasil operasi opening

8. Untuk melakukan operasi **closing,** gunakan syntax berikut.

```
closing = cv2.morphologyEx(im, cv2.MORPH_CLOSE, kernel)
```

Hasil syntax tersebut ditunjukkan pada Gambar 5.



Gambar 5. Hasil operasi closing

9. Untuk melakukan operasi **tophat,** gunakan syntax berikut.

```
tophat = cv2.morphologyEx(im, cv2.MORPH_TOPHAT, kernel)
```

Hasil syntax tersebut ditunjukkan pada Gambar 6.

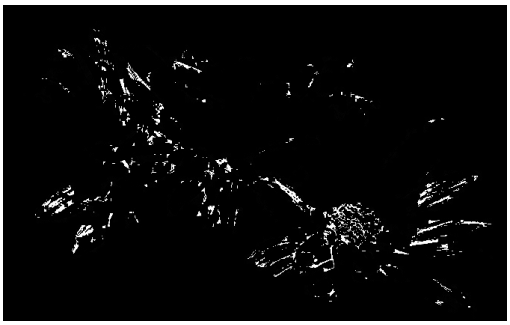


Gambar 6. Hasil operasi tophat

10. Untuk melakukan operasi **blackhat,** gunakan syntax berikut.

```
blackhat = cv2.morphologyEx(im, cv2.MORPH_BLACKHAT, kernel)
```

11. Hasil syntax tersebut ditunjukkan pada Gambar 7.

Gambar 7. Hasil operasi blackhat

Morphology image processing Scikit-Image

12. Import beberapa libraries

```
from skimage import morphology, filters
from skimage import data
```

13. Patikan **gambar sudah dalam format binary.** Jika gambar belum dalam format binary, tambahkan syntax berikut pada gambar yang sebelumnya sudah diubah menjadi black and white.

```
im = mpimg.imread('bw_flower.jpg')
binary = im > filters.threshold_otsu(im)
np.unique(binary)
```

14. Untuk melakukan operasi **skeletonize,** gunakan syntax berikut.

```
skeleton = morphology.skeletonize(binary)
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 12),
                         sharex=True, sharey=True)

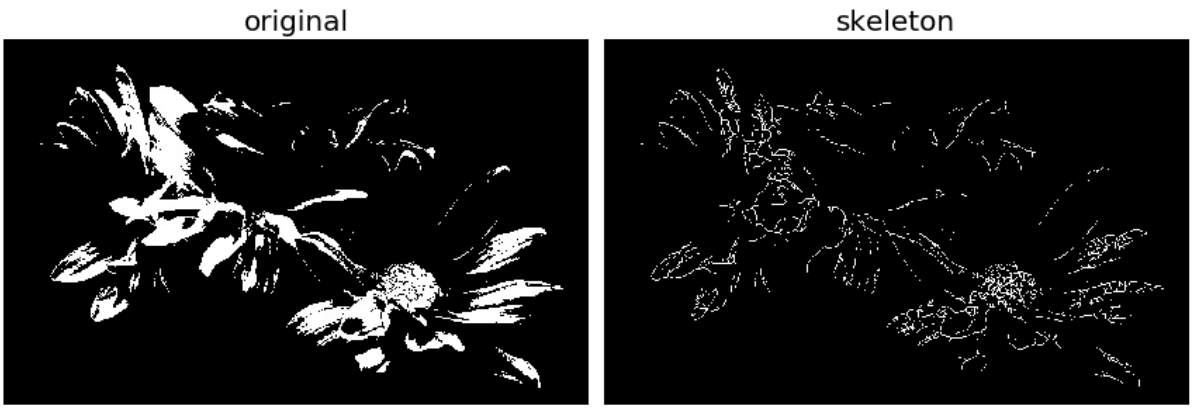
ax = axes.ravel()

ax[0].imshow(im)
ax[0].axis('off')
ax[0].set_title('original', fontsize=20)

ax[1].imshow(skeleton)
ax[1].axis('off')
ax[1].set_title('skeleton', fontsize=20)

fig.tight_layout()
plt.show()
```

15. Hasil syntax tersebut ditunjukkan pada Gambar 8.



Gambar 8. Hasil operasi skeletonize

16. Untuk melakukan operasi **thinning,** gunakan syntax berikut.

```
thinned = morphology.thin(binary)
thinned_partial = morphology.thin(binary, max_iter=25)
```

17. Untuk menampilkan 4 gambar sekaligus, tambahkan syntax berikut.

```
fig, axes = plt.subplots(2, 2, figsize=(8, 8), sharex=True, sharey=True)
ax = axes.ravel()

ax[0].imshow(binary, cmap=plt.cm.gray)
ax[0].set_title('original')
ax[0].axis('off')

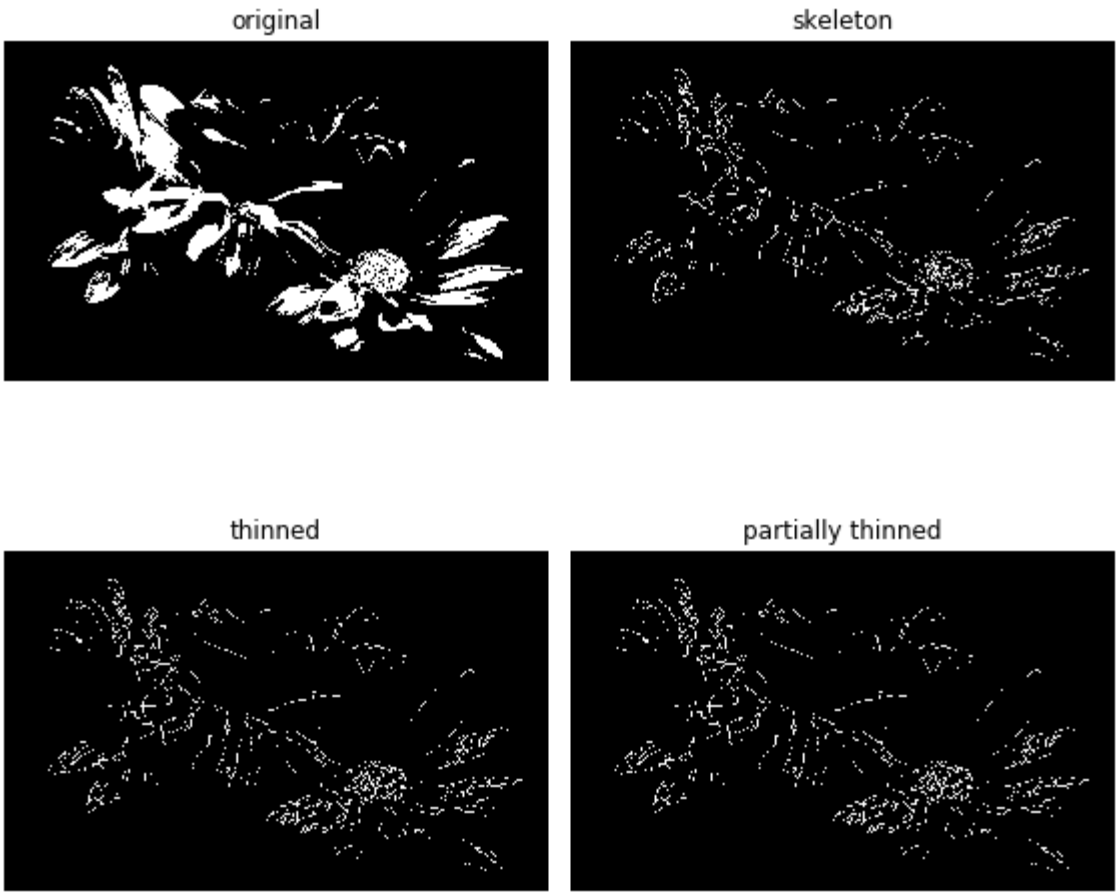
ax[1].imshow(skeleton, cmap=plt.cm.gray)
ax[1].set_title('skeleton')
ax[1].axis('off')

ax[2].imshow(thinned, cmap=plt.cm.gray)
ax[2].set_title('thinned')
ax[2].axis('off')

ax[3].imshow(thinned_partial, cmap=plt.cm.gray)
ax[3].set_title('partially thinned')
ax[3].axis('off')

fig.tight_layout()
plt.show()
```

18. Hasil syntax berikut ditujukkan pada Gambar 9.



Gambar 9. Hasil operasi thinning

19. Untuk melakukan operasi **convex hull,** gunakan syntax berikut.

```
convexhull = morphology.convex_hull_image(binary)
```

Hasil dari syntax tersebut ditunjukkan pada Gambar 10.



Gambar 10. Hasil operasi thinning

REFERENSI

1. Sandipan Dey, Hands on Image Processing with Phyton, Packt Publishing, 2018, **Chapter 6**
2. docs.opencv.org
3. scikit.image.org

MODUL 5
IMAGE SEGMENTATION

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari beberapa metode segmentasi dalam pengolahan gambar

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan metode-metode segmentasi dalam pengolahan gambar dengan Python (C_3)

PENUNJANG PRAKTIKUM

1. Jupyter Lab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Image Segmentation using Skicit-Image

1. Import beberapa libraries

```
import numpy as np
import matplotlib.pyplot as plt

from skimage import data
from skimage.exposure import histogram
```

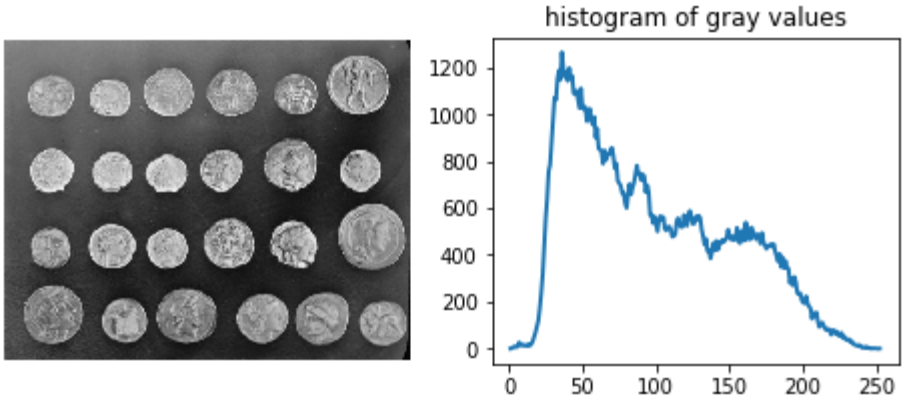
2. Plot histogram data coins dari library skimage.data

```
coins = data.coins()
hist, hist_centers = histogram(coins)

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].imshow(coins, cmap=plt.cm.gray)
axes[0].axis('off')
axes[1].plot(hist_centers, hist, lw=2)
axes[1].set_title('histogram of gray values')
```

3. Hasil syntax tersebut ditunjukkan pada Gambar 1.

```
Text(0.5, 1.0, 'histogram of gray values')
```



Gambar 1. Hasil plot histogram

4. Lakukan **Thresholding** pada gambar dengan syntax berikut.

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3), sharey=True)

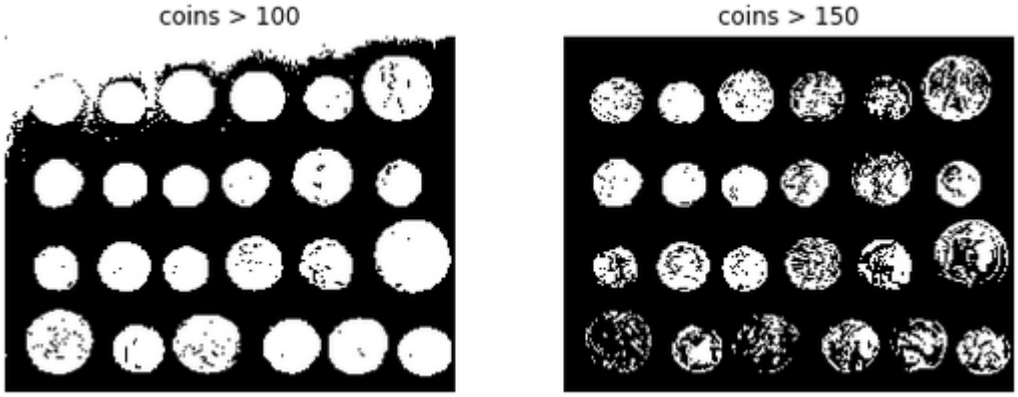
axes[0].imshow(coins > 100, cmap=plt.cm.gray)
axes[0].set_title('coins > 100')

axes[1].imshow(coins > 150, cmap=plt.cm.gray)
axes[1].set_title('coins > 150')

for a in axes:
    a.axis('off')

plt.tight_layout()
```

5. Hasil syntax tersebut ditunjukkan pada Gambar 2.



Gambar 2. Hasil Thresholding

6. Threshold juga dapat dilakukan dengan metode otsu threshold seperti pada syntax berikut.

```
from skimage import data
from skimage import filters
from skimage import exposure
import matplotlib.pyplot as plt

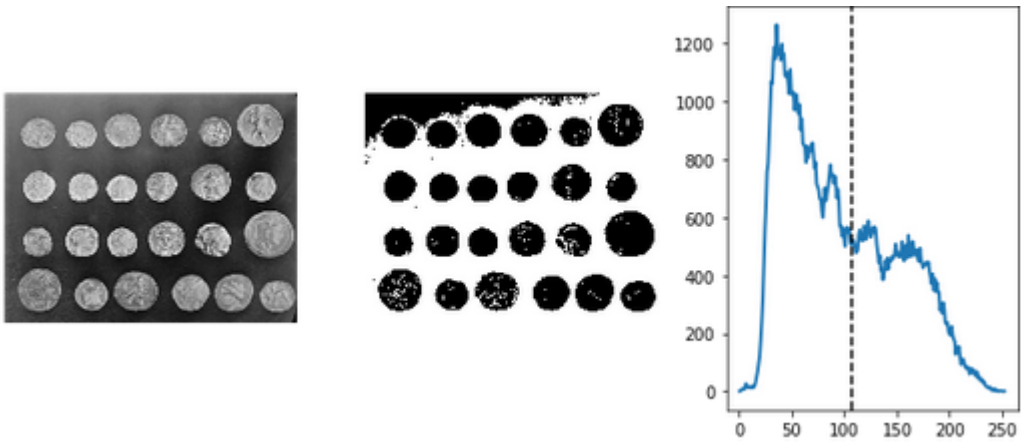
coins = data.coins()
val = filters.threshold_otsu(coins)

hist, bins_center = exposure.histogram(coins)

plt.figure(figsize=(9, 4))
plt.subplot(131)
plt.imshow(coins, cmap='gray', interpolation='nearest')
plt.axis('off')
plt.subplot(132)
plt.imshow(coins < val, cmap='gray', interpolation='nearest')
plt.axis('off')
plt.subplot(133)
plt.plot(bins_center, hist, lw=2)
plt.axvline(val, color='k', ls='--')

plt.tight_layout()
plt.show()
```

7. Hasil syntax tersebut ditunjukkan pada Gambar 3.



Gambar 3. Hasil Otsu Threshold

Edge Based Segmentation

Canny Edge

8. Untuk melakukan segmentasi dengan edge based, gunakan metode canny dengan menambahkan syntax berikut.

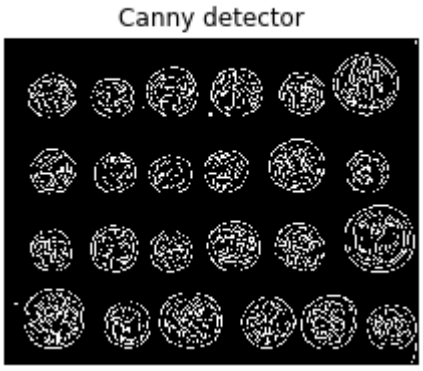
```
from skimage.feature import canny


edges = canny(coins/255.)

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(edges, cmap=plt.cm.gray)
ax.set_title('Canny detector')
ax.axis('off')
```

9. Hasil syntax tersebut ditunjukkan pada Gambar 4.

```
(-0.5, 383.5, 302.5, -0.5)
```



Gambar 4. Hasil Canny Edge

Filling the holes

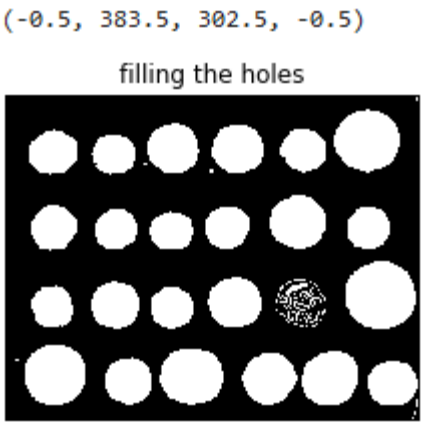
10. Contour tersebut kemudian "diisi" dengan metode morphology sesuai dengan syntax berikut.

```
from scipy import ndimage as ndi

fill_coins = ndi.binary_fill_holes(edges)

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(fill_coins, cmap=plt.cm.gray)
ax.set_title('filling the holes')
ax.axis('off')
```

11. Hasil syntax tersebut ditunjukkan pada Gambar 5.



Gambar 5. Hasil filling the holes

Removing Small Objects

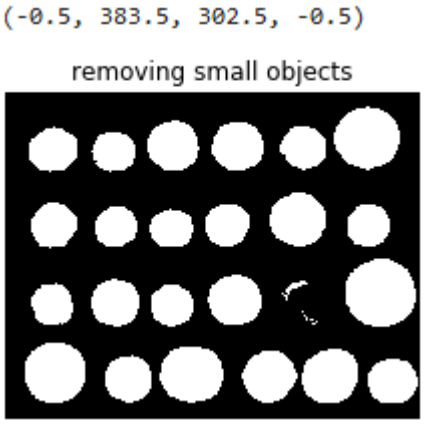
12. Untuk menghilangkan objek palsu yang tidak diperlukan tentukan ukuran minimum objek yang valid menggunakan syntax berikut.

```
from skimage import morphology

coins_cleaned = morphology.remove_small_objects(fill_coins, 21)

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(coins_cleaned, cmap=plt.cm.gray)
ax.set_title('removing small objects')
ax.axis('off')
```

13. Hasil syntax tersebut ditunjukkan pada Gambar 6.



Gambar 6. Hasil Removing Small Objects

Region Based Segmentation

Elevation Map

14. Tentukan elevation map menggunakan Sobel gradient dengan syntax berikut.

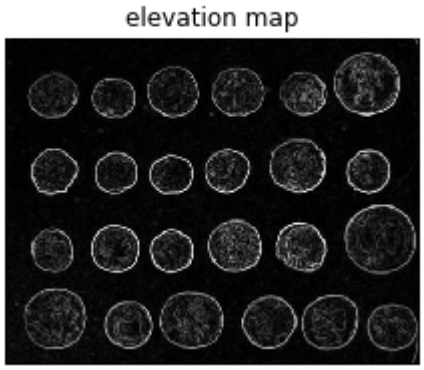
```
from skimage.filters import sobel

elevation_map = sobel(coins)

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(elevation_map, cmap=plt.cm.gray)
ax.set_title('elevation map')
ax.axis('off')
```

15. Hasil syntax tersebut ditunjukkan pada Gambar 7.

```
(-0.5, 383.5, 302.5, -0.5)
```



Gambar 7. Hasil Elevation Map

Markers

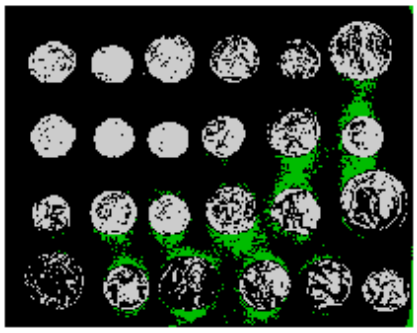
16. Tentukan markers untuk background dan objek koin berdasarkan bagian ekstrim dari histogram dengan gray level menggunakan syntax berikut.

```
markers = np.zeros_like(coins)
markers[coins < 30] = 1
markers[coins > 150] = 2

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(markers, cmap=plt.cm.nipy_spectral)
ax.set_title('markers')
ax.axis('off')
```

Hasil syntax tersebut ditunjukkan pada Gambar 8.

```
(-0.5, 383.5, 302.5, -0.5)
```



Gambar 8. Hasil Markers

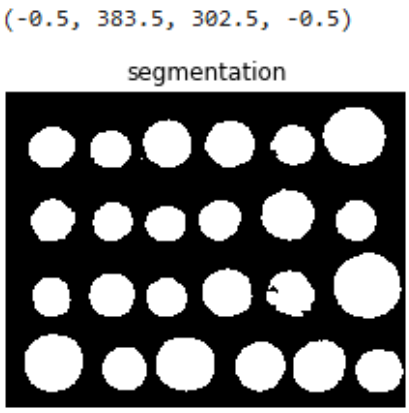
Watershed Transform

17. Gunakan Watershed Transform dengan menambahkan syntax berikut.

```
segmentation = morphology.watershed(elevation_map, markers)

fig, ax = plt.subplots(figsize=(4, 3))
ax.imshow(segmentation, cmap=plt.cm.gray)
ax.set_title('segmentation')
ax.axis('off')
```

18. Hasil syntax tersebut ditunjukkan pada Gambar 9.



Gambar 9. Hasil Watershed Transform

19. Untuk hasil yang lebih baik, yaitu masing-masing koin tersegmentasi dan terlabel dapat digunakan syntax berikut.

```
from skimage.color import label2rgb

segmentation = ndi.binary_fill_holes(segmentation - 1)
labeled_coins, _ = ndi.label(segmentation)
image_label_overlay = label2rgb(labeled_coins, image=coins)

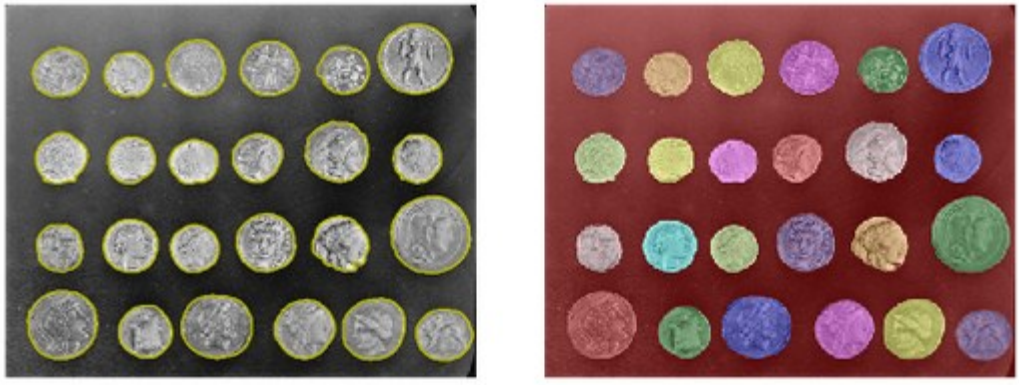
fig, axes = plt.subplots(1, 2, figsize=(8, 3), sharey=True)
axes[0].imshow(coins, cmap=plt.cm.gray)
axes[0].contour(segmentation, [0.5], linewidths=1.2, colors='y')
axes[1].imshow(image_label_overlay)

for a in axes:
    a.axis('off')

plt.tight_layout()

plt.show()
```

Hasil syntax tersebut ditunjukkan pada Gambar 10.



Gambar 10. Hasil segmentasi dan labelling

Image Segmentation using OpenCV

20. Gunakan Otsu binarization untuk memperkirakan objek dari koin menggunakan syntax berikut.

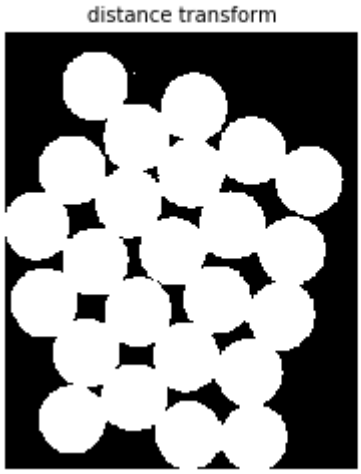
```
import cv2

img = cv2.imread('coins.jpg')
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(gray,0,255,cv.THRESH_BINARY_INV+cv2.THRESH_OTSU)

plt.imshow(thresh, cmap=plt.cm.gray)
```

Hasil syntax tersebut ditunjukkan pada Gambar 11.

```
Text(0.5, 1.0, 'distance transform')
```



Gambar 11. Hasil Otsu binarization

21. Untuk noise removal dan Thresholding gunakan syntax berikut.

```
# noise removal
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)

# sure background area
sure_bg = cv2.dilate(opening,kernel,iterations=3)

# Finding sure foreground area
dist_transform = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, sure_fg = cv2.threshold(dist_transform,0.7*dist_transform.max(),255,0)

# Finding unknown region
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg,sure_fg)
```

22. . Lakukan plot gambar dengan syntax berikut.

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(12, 12),
                         sharex=True, sharey=True)

ax = axes.ravel()

ax[0].imshow(opening, cmap=plt.cm.gray)
ax[0].axis('off')
ax[0].set_title('opening', fontsize=20)

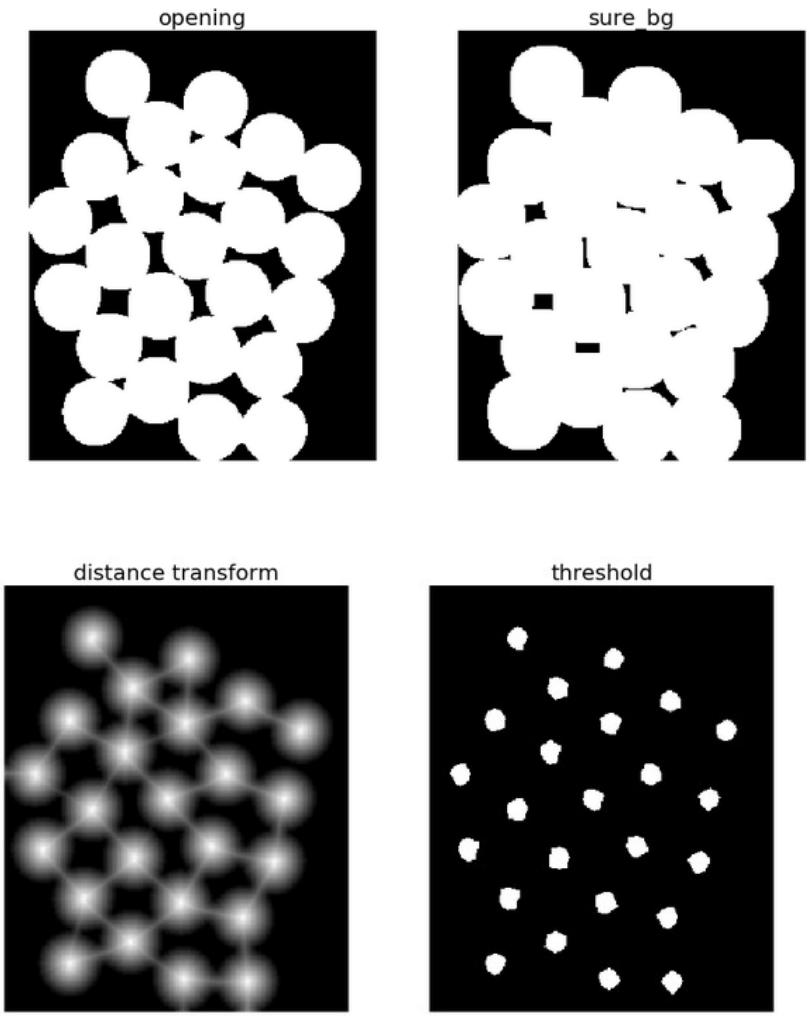
ax[1].imshow(sure_bg, cmap=plt.cm.gray )
ax[1].axis('off')
ax[1].set_title('sure_bg', fontsize=20)

ax[2].imshow(dist_transform, cmap=plt.cm.gray)
ax[2].axis('off')
ax[2].set_title('distance transform', fontsize=20)

ax[3].imshow(sure_fg, cmap=plt.cm.gray)
ax[3].axis('off')
ax[3].set_title('threshold', fontsize=20)

fig.tight_layout()
plt.show()
```

Hasil kedua syntax tersebut ditunjukkan pada Gambar 12.



Gambar 12. Hasil noise removal dan thresholding

23. Lakukan marker labelling dengan syntax berikut.

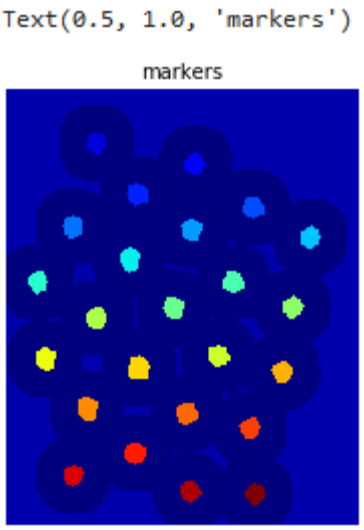
```
# Marker labelling
ret, markers = cv.connectedComponents(sure_fg)

# Add one to all labels so that sure background is not 0, but 1
markers = markers+1

# Now, mark the region of unknown with zero
markers[unknown==255] = 0

plt.imshow(markers,cmap=plt.cm.jet)
```

Hasil syntax tersebut ditunjukkan pada Gambar 13.

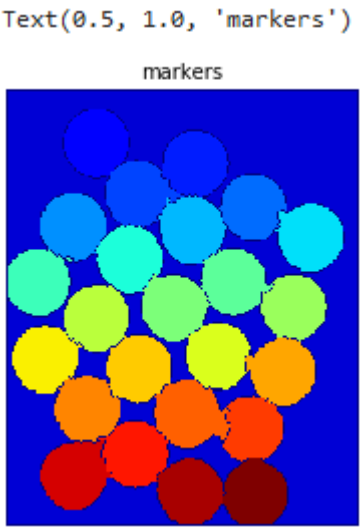


Gambar 13. Hasil Markers Labelling

24. Lakukan metode wathershed dengan syntax berikut.

```
markers = cv2.watershed(img,markers)
img[markers == -1] = [255,0,0]
plt.imshow(markers,cmap=plt.cm.jet )
plt.axis('off')
plt.title('markers', fontsize=10)
```

Hasil dari syntax tersebut ditunjukkan pada Gambar 14.



Gambar 14. Hasil markers watershed

REFERENSI

1. Sandipan Dey, Hands on Image Processing with Phyton, Packt Publishing, 2018, **Chapter 8**
2. docs.opencv.org
3. scikit.image.org

MODUL 6
FEATURE DETECTION & DESCRIPTION

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari beberapa metode feature detection dan feature description

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan metode-metode feature detection dalam pengolahan gambar dengan Python (C_3)

PENUNJANG PRAKTIKUM

1. Jupyter Lab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Harris Corner Detection

Metode Harris Corner Detection dapat diimplementasikan dengan syntax **cv.cornerHarris().** Argumen yang digunakan adalah

img – input image (dalam format *grayscale / float32)*.
blockSize – ukuran yang digunakan untuk corner detection
ksize – Aperture parameter dari Sobel derivative yang digunakan
k – parameter Harris detector.

Implementasi harris corner detection dapat dilakukan dengan cara sebagai berikut.

1. Import beberapa libraries

```
import cv2
import numpy as np
```

2. Siapkan gambar. Dalam contoh ini digunakan gambar "chessboard,jpg". Lalu lakukan implementasi dengan syntax berikut.

```
img = cv2.imread('chessboard.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

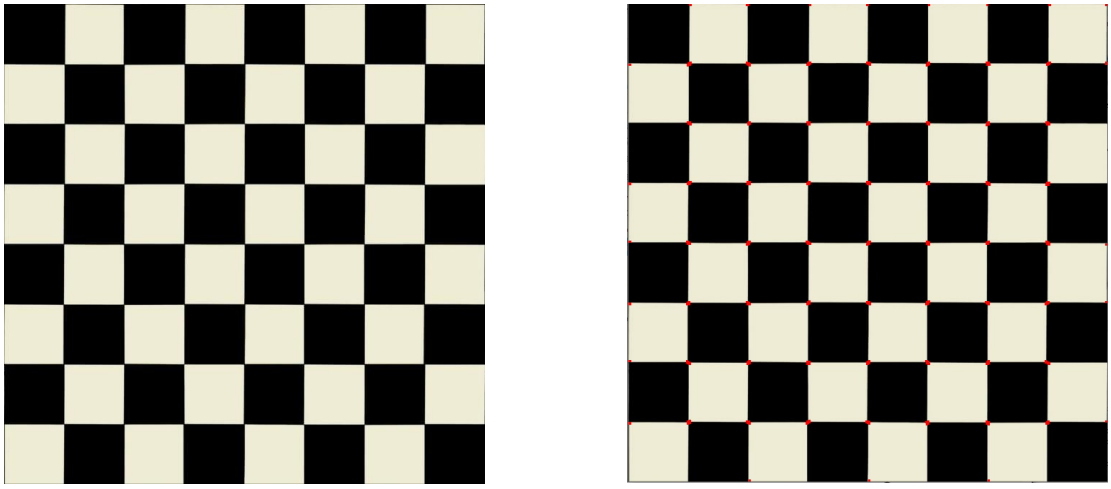
gray = np.float32(gray)
dst = cv2.cornerHarris(gray,2,3,0.04)

# Result is dilated for making the corners, not important
dst = cv2.dilate(dst,None)

# Threshold for an optimal value, it may vary depending on the image
img[dst>0.01*dst.max()]=[0,0,255]

cv2.imwrite('chessboard_harris.jpg',img)
```

Hasil syntax tersebut ditunjukkan pada Gambar 1. Gambar kiri merupakan gambar input sedangkan gambar kanan merupakan gambar output.



Input Output

Gambar 1. Hasil Harris Corner Detection

SIFT (Scale-Invariant Feature Transform)

Metode SIFT dapat diimplementasikan dengan syntax **sift = cv2.xfeatures2d.SIFT_create().**
Contoh implementasi metode SIFT adalah sebagai berikut.

3. Siapkan gambar. Dalam contoh ini digunakan gambar "home.jpg". Lalu lakukan implementasi dengan syntax berikut.

```
img2 = cv2.imread('home.jpg')
gray= cv2.cvtColor(img2,cv2.COLOR_BGR2GRAY)
```

```
sift = cv2.xfeatures2d.SIFT_create()
kp = sift.detect(gray,None)
```

```
img3 =cv2.drawKeypoints(gray,kp,img2)
cv2.imwrite('sift_keypoints.jpg',img3)
```

Hasil syntax tersebut ditunjukkan pada Gambar 2.



Input Output

Gambar 2. Hasil implementasi metode SIFT

SURF (Speeded Up Robust Features)

Metode SURF dapat diimplementasikan dengan syntax **surf = cv2.xfeatures2d.SURF_create().**
Contoh implementasi metode SURF adalah sebagai berikut.

4. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg". Lalu lakukan implementasi dengan syntax berikut.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
img = cv2.imread('butterfly.jpg',0)
# Create SURF object. You can specify params here or later.
# Here I set Hessian Threshold to 400
surf = cv2.xfeatures2d.SURF_create(400)

# Find keypoints and descriptors directly
kp, des = surf.detectAndCompute(img,None)
len(kp)
```

```
# Check present Hessian threshold
print( surf.getHessianThreshold() )
```

```
# We set it to some 50000. Remember, it is just for representing in picture.
# In actual cases, it is better to have a value 300-500
surf.setHessianThreshold(50000)

# Again compute keypoints and check its number.
kp, des = surf.detectAndCompute(img,None)
print( len(kp) )
```

```
img2 = cv2.drawKeypoints(img,kp,None,(255,0,0),4)
plt.imshow(img2),plt.show()
```

Hasil syntax tersebut ditunjukkan pada Gambar 3.



Input



Output

Gambar 2. Hasil implementasi metode SURF

FAST Algorithms

Metode FAST dapat diimplementasikan dengan syntax **fast = cv2.FastFeatureDetector_create()**
Contoh implementasi metode FAST adalah sebagai berikut.

5. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg". Lalu lakukan implementasi dengan syntax berikut.

```
img3 = cv2.imread('butterfly.jpg',0)
# Initiate FAST object with default values
fast = cv2.FastFeatureDetector_create()
# find and draw the keypoints
kp = fast.detect(img3,None)
img4 = cv2.drawKeypoints(img3, kp, None, color=(255,0,0))

# Print all default params
print( "Threshold: {}".format(fast.getThreshold()) )
print( "nonmaxSuppression:{}".format(fast.getNonmaxSuppression()) )
print( "neighborhood: {}".format(fast.getType()) )
print( "Total Keypoints with nonmaxSuppression: {}".format(len(kp)) )
cv2.imwrite('fast_true.png',img4)

# Disable nonmaxSuppression
fast.setNonmaxSuppression(0)
kp = fast.detect(img3,None)
print( "Total Keypoints without nonmaxSuppression: {}".format(len(kp)) )
img5 = cv2.drawKeypoints(img3, kp, None, color=(255,0,0))
cv2.imwrite('fast_false.png',img5)
```

Hasil syntax tersebut ditunjukkan pada Gambar 3.

```
Threshold: 10
nonmaxSuppression:True
neighborhood: 2
Total Keypoints with nonmaxSuppression: 4376
Total Keypoints without nonmaxSuppression: 324
```

Hasil output "print"



Input



Output (fast_true)



Output(fast_false)

Gambar 3. Hasil implementasi metode FAST

BRIEF (Binary Robust Independent Elementary Features)

Metode BRIEF dapat diimplementasikan dengan syntax **brief = cv2.xfeatures2d.BriefDescriptorExtractor_create()**

Contoh implementasi metode BRIEF adalah sebagai berikut.

6. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg". Lalu lakukan implementasi dengan syntax berikut.

```
img = cv2.imread('butterfly.jpg',0)
# Initiate FAST detector
star = cv2.xfeatures2d.StarDetector_create()

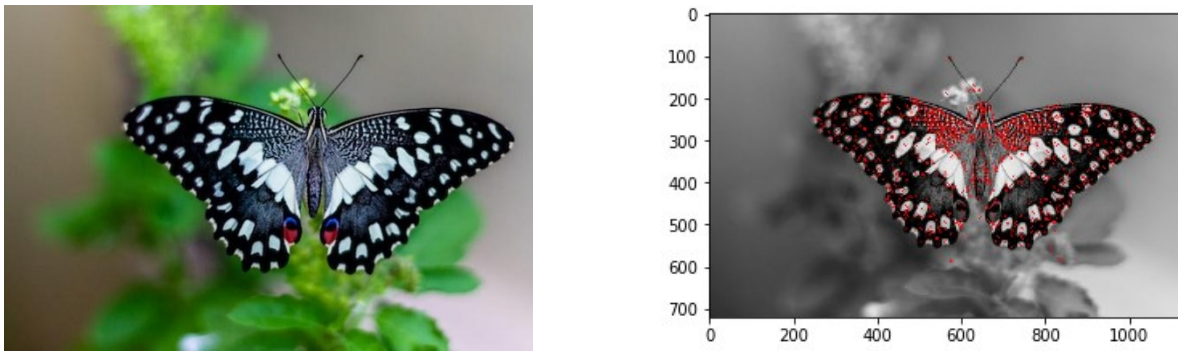
# Initiate BRIEF extractor
brief = cv2.xfeatures2d.BriefDescriptorExtractor_create()

# find the keypoints with STAR
kp = star.detect(img,None)

# compute the descriptors with BRIEF
kp, des = brief.compute(img, kp)
print( brief.descriptorSize() )
print( des.shape )
```

```
img2 = cv2.drawKeypoints(img,kp,None,(255,0,0))
plt.imshow(img2),plt.show()
```

Hasil syntax tersebut ditunjukkan pada Gambar 4.



Input Output

Gambar 4. Hasil implementasi metode BRIEF

ORB (Oriented Fast and Rotated BRIEF)

Metode ORB dapat diimplementasikan dengan syntax **orb = cv.ORB_create()**

Contoh implementasi metode ORB adalah sebagai berikut.

7. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg". Lalu lakukan implementasi dengan syntax berikut.

```
img3 = cv2.imread('BUTTERFLY.jpg',0)

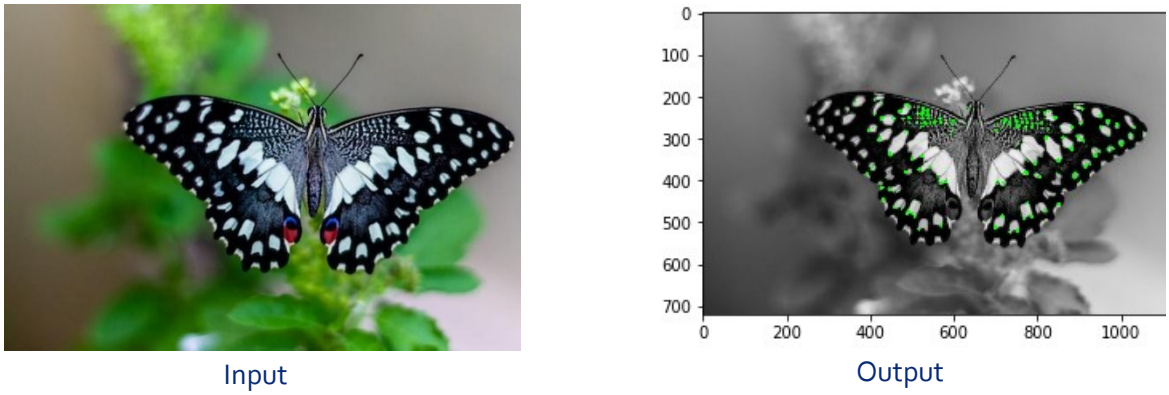
# Initiate ORB detector
orb = cv.ORB_create()

# find the keypoints with ORB
kp = orb.detect(img3,None)

# compute the descriptors with ORB
kp, des = orb.compute(img3, kp)

# draw only keypoints location,not size and orientation
img4 = cv.drawKeypoints(img3, kp, None, color=(0,255,0), flags=0)
plt.imshow(img4), plt.show()
```

Hasil syntax tersebut ditunjukkan pada Gambar 5.



Input Output

Gambar 5. Hasil implementasi metode ORB

Brute Force Matching with ORB

Contoh implementasi metode brute force matching adalah sebagai berikut.

8. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg" dan "butterfly2.jpg yang merupakan hasil crop dari gambar butterfly.jpg. Selanjutnya, lakukan implementasi dengan syntax berikut.

```
img1 = cv2.imread('butterfly2.jpg',cv2.IMREAD_GRAYSCALE) # queryImage
img2 = cv2.imread('butterfly.jpg',cv2.IMREAD_GRAYSCALE) # trainImage

# Initiate ORB detector
orb = cv2.ORB_create()

# find the keypoints and descriptors with ORB
kp1, des1 = orb.detectAndCompute(img1,None)
kp2, des2 = orb.detectAndCompute(img2,None)
```

```
# create BFMatcher object
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
# Match descriptors.
matches = bf.match(des1,des2)
# Sort them in the order of their distance.
matches = sorted(matches, key = lambda x:x.distance)
# Draw first 10 matches.
img3 = cv2.drawMatches(img1,kp1,img2,kp2,matches[:10],None,flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
plt.imshow(img3),plt.show()
```

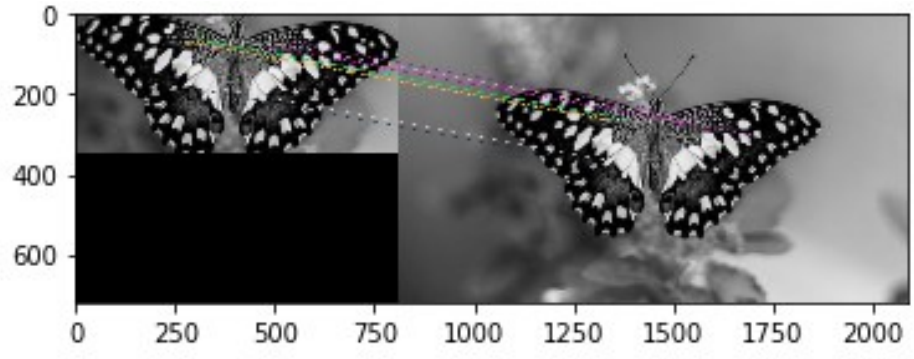
Hasil syntax tersebut ditunjukkan pada Gambar 7.



Butterfly.jpg



Butterfly2.jpg



Output

Gambar 6. Hasil implementasi Brute Force Matching

FLANN based Matcher

Contoh implementasi metode FLANN based Matcher adalah sebagai berikut.

9. Siapkan gambar. Dalam contoh ini digunakan gambar "butterfly.jpg" dan "butterfly2.jpg yang merupakan hasil crop dari gambar butterfly.jpg. Selanjutnya, lakukan implementasi dengan syntax berikut.

```
img3 = cv2.imread('butterfly2.jpg',cv2.IMREAD_GRAYSCALE)     # queryImage
img4 = cv2.imread('butterfly.jpg',cv2.IMREAD_GRAYSCALE) # trainImage

# Initiate SIFT detector
sift = cv2.xfeatures2d.SIFT_create()
# find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(img3,None)
kp2, des2 = sift.detectAndCompute(img4,None)

# FLANN parameters
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks=50)   # or pass empty dictionary
flann = cv2.FlannBasedMatcher(index_params,search_params)
matches = flann.knnMatch(des1,des2,k=2)

# Need to draw only good matches, so create a mask
matchesMask = [[0,0] for i in range(len(matches))]
# ratio test as per Lowe's paper
for i,(m,n) in enumerate(matches):
    if m.distance < 0.7*n.distance:
        matchesMask[i]=[1,0]
draw_params = dict(matchColor = (0,255,0),
                   singlePointColor = (255,0,0),
                   matchesMask = matchesMask,
                   flags = cv2.DrawMatchesFlags_DEFAULT)
img5 = cv2.drawMatchesKnn(img3,kp1,img4,kp2,matches,None,**draw_params)
plt.imshow(img5,),plt.show()
```

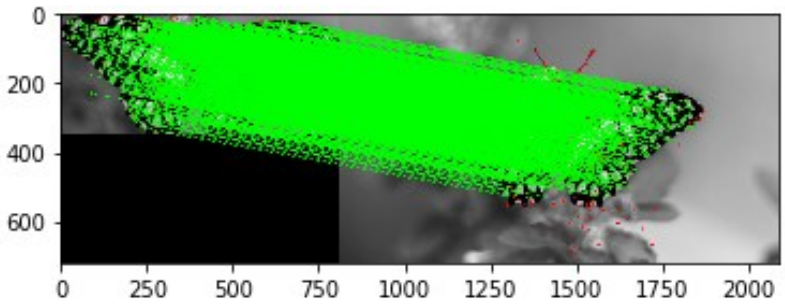
Hasil syntax tersebut ditunjukkan pada Gambar 7.



Butterfly.jpg



Butterfly2.jpg



Output

Gambar 7. Hasil implementasi metode FLANN based matcher

REFERENSI

1. docs.opencv.org
2. medium.com

MODUL 7
SUPERVISED LEARNING UNTUK ANALISIS GAMBAR

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari dua metode *supervised learning* yaitu k-neirest neighbour untuk analisis gambar

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menguji metode-metode supervised learning dalam analisis gambar dengan Python (C_4)

PENUNJANG PRAKTIKUM

1. Jupyter Lab/Google Colab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

K-Nearest Neigbour for image classification

1. Install library vega dengan mengetikkan 'pip install vega' pada terminal
2. Import beberapa library berikut.

```
import sys
sys.executable
```

```
import pandas as pd
import numpy as np
from vega import VegaLite
from IPython.display import clear_output
```

3. Generate cluster dengan syntax berikut.

```
# %load gen_points.py
import numpy as np
import random
import math

def gen_points(val_seed_xy=(80, 100), num_class=2, num_points=100, global_std=20):
    # cluster center
    centers = []
    for i in range(num_class):
        centers.append((val_seed_xy[0] * random.random(), val_seed_xy[1] * random.random()))

    # Points near centers
    X = np.empty(shape=[0, 2])
    y = np.empty(shape=[0, 1])
    for i in range(num_points):
        cls = num_class * random.random()
        cls = math.floor(cls)
        xcord = centers[cls][0] + global_std * random.random()
        ycord = centers[cls][1] + global_std * random.random()
        X = np.append(X, [[xcord, ycord]], axis=0)
        y = np.append(y, [[cls]], axis=0)

    return (X, y)
```

```
(X, y) = gen_points()
```

```
columns = ["x", "y", "c"]
mat = np.hstack((X, y))
df = pd.DataFrame(mat, columns=columns)
df.head()
```

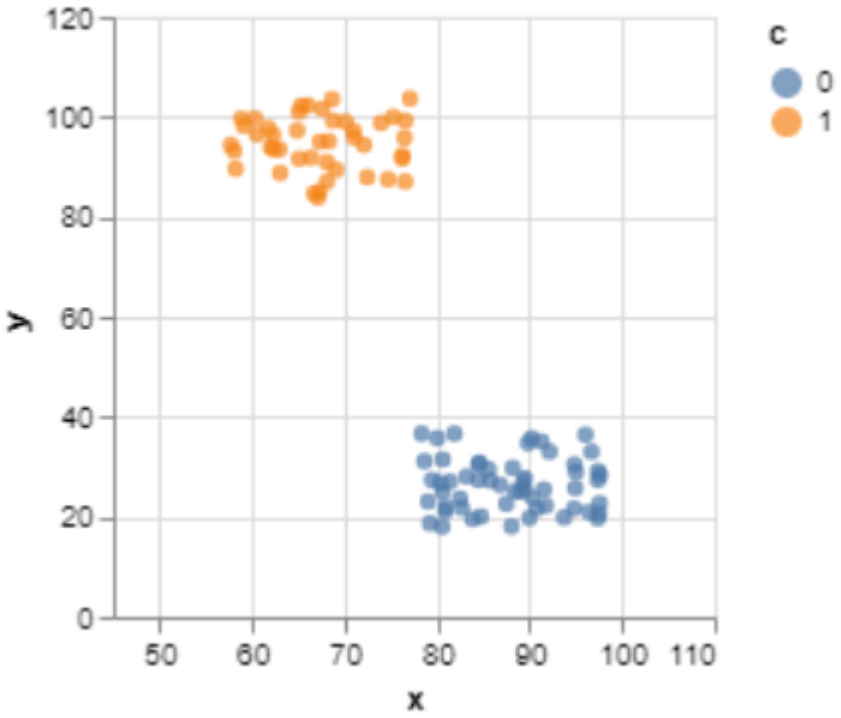
Ouput

	x	y	c
0	60.320741	99.715404	1.0
1	80.600698	25.070080	0.0
2	82.688038	21.790847	0.0
3	84.595532	30.880273	0.0
4	97.394249	29.187617	0.0

4. Visualisasi cluster dengan syntax berikut.

```
VegaLite({
    "mark": { "type": "point", "filled": "true" },
    "encoding": {
        "y": {
            "type": "quantitative",
            "field": "y",
            "scale": {
                "domain": [df['y'].min() - 10, df['y'].max() + 10]
            }
        },
        "x": {
            "type": "quantitative",
            "field": "x",
            "scale": {
                "domain": [df['x'].min() - 10, df['x'].max() + 10]
            }
        },
        "color": { "type": "nominal", "field": "c" }
    }
}, df)
```

Output



5. Jika ingin menyimpan data, gunakan syntax berikut.

```
df.to_csv('gen-data.csv', index=False)
```

6. Terapkan algoritma pada dataset real

```python
# %load knn_algo.py
import numpy as np

def default_progress_fn(i, total):
    pass

class KNN:
    def __init__ (self, k, progress_fn=default_progress_fn):
        """ Pass k as hyperparameter"""
        self.k = k
        self.progress_fn = progress_fn

    def train(self, X, y):
        """
            X is example training matrix. Every row of X contains one training example. Each training example
            may have `d` features. If there are `m` such examples then X is `m x d` matrix.
            y is the label matrix corrosponding to each training example. Hence y is `m x 1` matrix.
        """

        # During training time stores the references of the input only
        self.tX = X
        self.ty = y
```

```
    def predict(self, X):
        """
            Predict y based on test data X.
        """

        num_training = X.shape[0]
        YPred = np.zeros(num_training, dtype = self.ty.dtype)

        for i in range(num_training):
            # Euclidean distance is used to find out distance between two datapoint.
            distances = np.reshape(np.sqrt(np.sum(np.square(self.tX - X[i, :]), axis=1)), (-1, 1))
            # Along with the distance stack the labels so that we can vote easily
            distance_label = np.hstack((distances, self.ty))
            # Simple majority voting based on the minimum distance
            sorted_distance = distance_label[distance_label[:,0].argsort()]
            k_sorted_distance = sorted_distance[:self.k,:]
            (labels, occurence) = np.unique(k_sorted_distance[:, 1], return_counts=True)
            label = labels[occurence.argsort()[0]]
            YPred[i] = label

            self.progress_fn(i, num_training)

        return YPred
```

```
import knn_algo as knnalgo
knn = knnalgo.KNN(3) # 3 Nearest neighbour
knn.train(X, y)
knn.predict(np.array([[10, 100], [10, 10]]))
```

Output

```
array([1., 0.])
```

7. Download data dari link http://www.cs.toronto.edu/~kriz/cifar.html

```
basedir_data = "./data/"
```

```
rel_path = basedir_data + "cifar-10-batches-py/"
```

8. Lakukan deserealize

```
def unpickle(file):
    import pickle
    with open(file, 'rb') as fo:
        dict = pickle.load(fo, encoding='bytes')
    return dict

X = unpickle(rel_path + 'data_batch_1')
img_data = X[b'data']
img_label_orig = img_label = X[b'labels']
img_label = np.array(img_label).reshape(-1, 1)
```

```
print(img_data)
print('shape', img_data.shape)
```

Output

```
[[ 59  43  50 ... 140  84  72]
 [154 126 105 ... 139 142 144]
 [255 253 253 ...  83  83  84]
 ...
 [ 71  60  74 ...  68  69  68]
 [250 254 211 ... 215 255 254]
 [ 62  61  60 ... 130 130 131]]
shape (10000, 3072)
```

9. Cek label dengan syntax berikut.

```
print(img_label)
print('shape', img_label.shape)
```

Output

```
[[6]
 [9]
 [9]
 ...
 [1]
 [1]
 [5]]
shape (10000, 1)
```

10. Load testing data, cek apakah data test memiliki bentuk gambar yang sama dengan training data

```
test_X = unpickle(rel_path + 'test_batch');
test_data = test_X[b'data']
test_label = test_X[b'labels']
test_label = np.array(test_label).reshape(-1, 1)
```

```
print(test_data)
print('shape', test_data.shape)
```

Output

```
[[158 159 165 ... 124 129 110]
 [235 231 232 ... 178 191 199]
 [158 158 139 ...   8   3   7]
 ...
 [ 20  19  15 ...  50  53  47]
 [ 25  15  23 ...  80  81  80]
 [ 73  98  99 ...  94  58  26]]
shape (10000, 3072)
```

11. Lakukan sampling 5 gambar dari data dan menampilkan output bentuk gambarnya.

```
sample_img_data = img_data[0:10, :]
print(sample_img_data)
print('shape', sample_img_data.shape)
```

```
[[ 59  43  50 ... 140  84  72]
 [154 126 105 ... 139 142 144]
 [255 253 253 ...  83  83  84]
 ...
 [ 28  30  33 ... 100  99  96]
 [134 131 128 ... 136 137 138]
 [125 110 102 ...  82  84  86]]
shape (10, 3072)
```

12. Dapatkan batch file sehingga nama gambar dapat ditampilkan

```
batch = unpickle(rel_path + 'batches.meta');
meta = batch[b'label_names']
print(meta)
```

```
[b'airplane', b'automobile', b'bird', b'cat', b'deer', b'dog', b'frog', b'horse', b'ship', b'truck']
```

13. Untuk menampilkan gambar gunakan syntax berikut.

```
# %load show_img.py
from PIL import Image
import numpy as np
from IPython.display import display

def default_label_fn(i, original):
    return original

def show_img(img_arr, label_arr, meta, index, label_fn=default_label_fn):
    """
        Given a numpy array of image from CIFAR-10 labels this method transform the data so that PIL can read and show
        the image.
        Check here how CIFAR encodes the image http://www.cs.toronto.edu/~kriz/cifar.html
    """

    one_img = img_arr[index, :]
    # Assume image size is 32 x 32. First 1024 px is r, next 1024 px is g, last 1024 px is b from the (r,g b) channel
    r = one_img[:1024].reshape(32, 32)
    g = one_img[1024:2048].reshape(32, 32)
    b = one_img[2048:]. reshape(32, 32)
    rgb = np.dstack([r, g, b])
    img = Image.fromarray(np.array(rgb), 'RGB')
    display(img)
    print(label_fn(index, meta[label_arr[index][0]].decode('utf-8')))
```

```
import show_img as si
for i in range(0, 10):
    si.show_img(sample_img_data, img_label, meta, i)
```



frog



truck



truck



deer



automobile



automobile



bird



horse



ship



cat

14. Jalankan algoritma KNN

```
data_point_no = 10
```

```
import time
def progress(i, total):
    clear_output(wait=True)
    print('completed ' + str(i + 1) + ' of total ' + str(total))
    time.sleep(10)

sample_test_data = test_data[:data_point_no, :]
knn = knnalgo.KNN(3, progress_fn=progress) # 3 Nearest neighbour
knn.train(img_data, img_label)
Ypred = knn.predict(sample_test_data)
```

```
completed 10 of total 10
```

```
def pred_label_fn(i, original):
    return original + '::' + meta[Ypred[i]].decode('utf-8')

for i in range(0, len(Ypred)):
    si.show_img(sample_test_data, test_label, meta, i, label_fn=pred_label_fn)
```



```
cat::airplane
```



```
ship::truck
```



```
ship::ship
```



```
airplane::bird
```



```
frog::cat
```



```
frog::frog
```



```
automobile::bird
```



```
frog::deer
```



```
cat::bird
```



```
automobile::bird
```

REFERENSI

1. docs.opencv.org
2. scipy-lectures.org
3. medium.com

MODUL 8
UNSUPERVISED LEARNING UNTUK ANALISIS GAMBAR

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mempelajari dua metode *unsupervised learning* yaitu k-means clustering dan PCA untuk analisis gambar

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menguji metode-metode unsupervised learning dalam analisis gambar dengan Python (C_4)

PENUNJANG PRAKTIKUM

1. Jupyter Lab/Google Colab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

K-means clustering for image color segmentation

Metode k-means clustering dapat diimplementasikan menggunakan library OpenCV dengan syntax **cv.kmeans()** input parameter syntax tersebut adalah sebagai berikut.

Samples – harus tipe np.float32

Nclusters (K) – jumlah cluster yang dibutuhkan

Criteria - kriteria termination. Ada beberapa parameter (type, max_iter, epsilon)

- Type
  1. cv.TERM_CRITERIA_EPS - memberhentikan agoritma jika nilai epsilon sudah terpenuhi
  2. cv.TERM_CRITERIA_MAX_ITER - memberhentikan agoritma jika jumlah iterasi sudah terpenuhi
  3. cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER - memberhentikan agoritma jika kedua kondisi epsilon dan jumlah iterasi sudah terpenuhi
- Max_iter : jumlah iterasi
- Epsilon : nilai akurasi yang diinginkan

Attemps

Flags - menentukan nilai pusat awal diambil. Ada dua jenis flag yang sering digunakan yaitu cv.KMEANS_PP_CENTERS dan cv.KMEANS_RANDOM_CENTERS.

Implementasi k-means clustering pada analisis gambar salah satunya adalah untuk melakukan fungsi segmentasi warna pada gambar. Implementasi tersebut dapat dilakukan dengan syntax berikut.

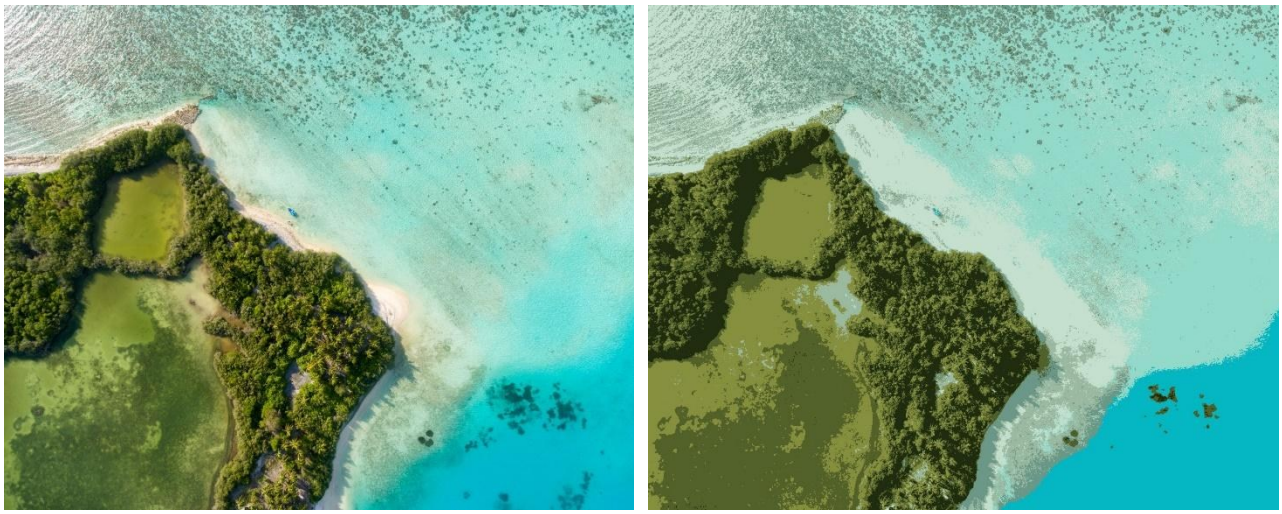
1. Implementasi syntax k-means clustering untuk segmentasi warna pada gambar

```python
import numpy as np
import cv2 as cv

img = cv.imread('panorama.jpeg')
Z = img.reshape((-1,3))
# convert to np.float32
Z = np.float32(Z)
# define criteria, number of clusters(K) and apply kmeans()
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 10, 1.0)
K = 8
ret,label,center=cv.kmeans(Z,K,None,criteria,10,cv.KMEANS_RANDOM_CENTERS)
# Now convert back into uint8, and make original image
center = np.uint8(center)
res = center[label.flatten()]
res2 = res.reshape((img.shape))

filename = 'kmeanssegmentation_res.jpeg'
cv.imwrite(filename, res2)
```

Hasil syntax tersebut ditunjukkan pada Gambar 1. Gambar kiri merupakan gambar input sedangkan gambar kanan merupakan gambar output.



Input Output

Gambar 1. Hasil k-means clustering untuk segmentasi warna (k=8)

2. Ubah nilai k dan amati perubahan output yang diperoleh.

K-means Clustering untuk Image Classification

K-means clustering juga dapat digunakan untuk proses klasifikasi gambar.

Contoh implementasi k-means clustering pada klasifikasi gambar dicontohkan pada syntax berikut,

3. Install library TensorFlow, Keras, dan scikit-learn(sklearn) dengan mengetikkan 'pip install tensorflow keras scikit-learn' pada terminal
4. Import MNIST dataset dari library keras. MNIST dataset merupakan dataset *benchmark* yang berisikan gambar angka 0 hingga 9 dengan ukuran 28x28 pixel

```
# Importing the dataset from keras
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print(type(x_train))
print(type(x_test))
print(type(y_train))
print(type(y_test))
```

Keluaran dari syntax di atas adalah sebagai berikut.

```
Using TensorFlow backend.
Downloading data from https://s3.amazonaws.com/img-datasets/mnist.npz
11493376/11490434 [==============================] - 2s 0us/step
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
```

Lakukan beberapa syntax di bawah untuk mengetahui beberapa karakteristik dataset.

```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Output

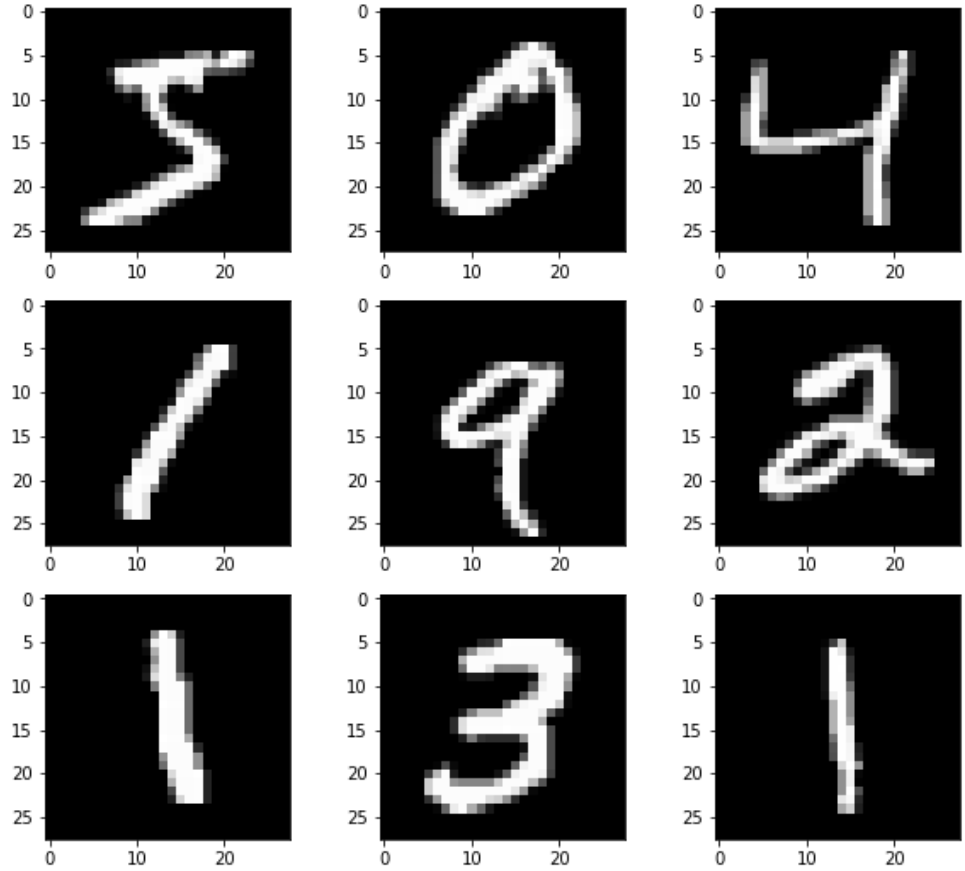
```
(60000, 28, 28)
(10000, 28, 28)
(60000,)
(10000,)
```

```
import matplotlib.pyplot as plt

plt.gray() # B/W Images
plt.figure(figsize = (10,9)) # Adjusting figure size
# Displaying a grid of 3x3 images
for i in range(9):
 plt.subplot(3,3,i+1)
 plt.imshow(x_train[i])
```

Output

<Figure size 432x288 with 0 Axes>



```
# Printing examples in 'y_train'
for i in range(5):
  print(y_train[i])
```

Output

```
5
0
4
1
9
```

5. Lakukan preprocessing data dengan beberapa syntax berikut.

```
# Checking the minimum and maximum values of x_train
print(x_train.min())
print(x_train.max())
```

Output

```
0
255
```

Karena dataset memiliki *range* nilai 0-255, maka data set tersebut perlu dinormalisasi sehingga setiap parameternya (dalam contoh ini pixel) memiliki distribusi data yang sama.

```
import numpy as np

# Data Normalization
# Conversion to float
x_train = x_train.astype(np.float32)
x_test = x_test.astype(np.float32)
# Normalization
x_train = x_train/255.0
x_test = x_test/255.0
```

```
# Checking the minimum and maximum values of x_train
print(x_train.min())
print(x_train.max())
```

Output

```
0.0
1.0
```

```
# Reshaping input data
X_train = x_train.reshape(len(x_train),-1)
X_test = x_test.reshape(len(x_test),-1)
```

```
# Checking the shapeprint(X_train.shape)
print(X_test.shape)
```

```
(10000, 784)
```

6. Buat model menggunakan mini batch k-means. Mini batch k-means memiliki prinsip yang sama dengan k-means, akan tetapi waktu komputasi yang diperlukan lebih sedikit.

```
from sklearn.cluster import MiniBatchKMeans
total_clusters = len(np.unique(y_test))
# Initialize the K-Means model
kmeans = MiniBatchKMeans(n_clusters = total_clusters)
# Fitting the model to training set
kmeans.fit(X_train)
kmeans.labels_
```

```
def retrieve_info(cluster_labels,y_train):
 # Initializing
 reference_labels = {}
 # For loop to run through each label of cluster label
 for i in range(len(np.unique(kmeans.labels_))):
   index = np.where(cluster_labels == i,1,0)
   num = np.bincount(y_train[index==1]).argmax()
   reference_labels[i] = num
 return reference_labels
```

```
reference_labels = retrieve_info(kmeans.labels_,y_train)
number_labels = np.random.rand(len(kmeans.labels_))
for i in range(len(kmeans.labels_)):
  number_labels[i] = reference_labels[kmeans.labels_[i]]
```

```
# Comparing Predicted values and Actual values
print(number_labels[:20].astype(int))
print(y_train[:20])
```

Output

```
[8 0 4 1 9 0 1 2 1 9 2 1 3 6 1 7 2 8 6 1]
[5 0 4 1 9 2 1 3 1 4 3 5 3 6 1 7 2 8 6 9]
```

```
# Calculating accuracy score
from sklearn.metrics import accuracy_score
print(accuracy_score(number_labels,y_train))
```

Output

```
0.5583666666666667
```

7. Karena akurasi masih sekitar 55%, lakukan optimisasi dengan syntax berikut.

```
# Function to calculate metrics for the model
from sklearn import metrics

def calculate_metrics(model,output):
  print('Number of clusters is {}'.format(model.n_clusters))
  print('Inertia : {}'.format(model.inertia_))
  print('Homogeneity : {}'.format(metrics.homogeneity_score(output,model.labels_)))

cluster_number = [10,16,36,64,144,256]
for i in cluster_number:
  total_clusters = len(np.unique(y_test))
  #Initialize the K-Means model
  kmeans = MiniBatchKMeans(n_clusters = i)
  # Fitting the model to training set
  kmeans.fit(X_train)
  # Calculating the metrics
  calculate_metrics(kmeans,y_train)
  # Calculating reference_labels
  reference_labels = retrieve_info(kmeans.labels_,y_train)
  # 'number_labels' is a list which denotes the number displayed in image
  number_labels = np.random.rand(len(kmeans.labels_))
  for i in range(len(kmeans.labels_)):
    number_labels[i] = reference_labels[kmeans.labels_[i]]
  print('Accuracy score : {}'.format(accuracy_score(number_labels,y_train)))
  print('\n')
```

Output

```
Number of clusters is 10
Inertia : 2433170.5
Homogeneity : 0.40725644291391216
Accuracy score : 0.48925


Number of clusters is 16
Inertia : 2211900.0
Homogeneity : 0.5449658094398563
Accuracy score : 0.6382833333333333

Number of clusters is 36
Inertia : 1954527.75
Homogeneity : 0.6914420105968607
Accuracy score : 0.7708666666666667


Number of clusters is 64
Inertia : 1819840.5
Homogeneity : 0.7288850577212039
Accuracy score : 0.7867


Number of clusters is 144
Inertia : 1631266.125
Homogeneity : 0.8078005460044674
Accuracy score : 0.8736333333333334


Number of clusters is 256
Inertia : 1514967.625
Homogeneity : 0.8406186370953125
Accuracy score : 0.8957666666666667
```

Karena akurasi tertinggi diperoleh ketika k=256, uji model dengan nilai k=256

```
# Testing model on Testing set
# Initialize the K-Means model
kmeans = MiniBatchKMeans(n_clusters = 256)
# Fitting the model to testing set
kmeans.fit(X_test)
# Calculating the metrics
calculate_metrics(kmeans,y_test)
# Calculating the reference_labels
reference_labels = retrieve_info(kmeans.labels_,y_test)
# 'number_labels' is a list which denotes the number displayed in image
number_labels = np.random.rand(len(kmeans.labels_))
for i in range(len(kmeans.labels_)):
  number_labels[i] = reference_labels[kmeans.labels_[i]]
print('Accuracy score : {}'.format(accuracy_score(number_labels,y_test)))
print('\n')
```

Output

```
Number of clusters is 256
Inertia : 247811.0
Homogeneity : 0.8474503827860905
Accuracy score : 0.8873
```

Skor akurasi yang diperoleh pada saat training dan pengujian memiliki hasil yang sangat mendekati yaitu sekitar 89%. Hal tersebut menunjukkan bahwa model tersebut tidak overfitting.

Eigenfaces using PCA

Penjelasan berikut menunjukkan implementasi PCA dan SVM untuk eigenfaces. Pada modul ini kita hanya akan mengimplementasikan preprocessing data menggunakan PCA. Untuk contoh yang lebih detail silakan akses di https://scipy-lectures.org/packages/scikit-learn/index.html#the-eigenfaces-example-chaining-pca-and-svms

8. Gunakan dataset dari library sklearn

```
from sklearn import datasets
faces = datasets.fetch_olivetti_faces()
faces.data.shape
```

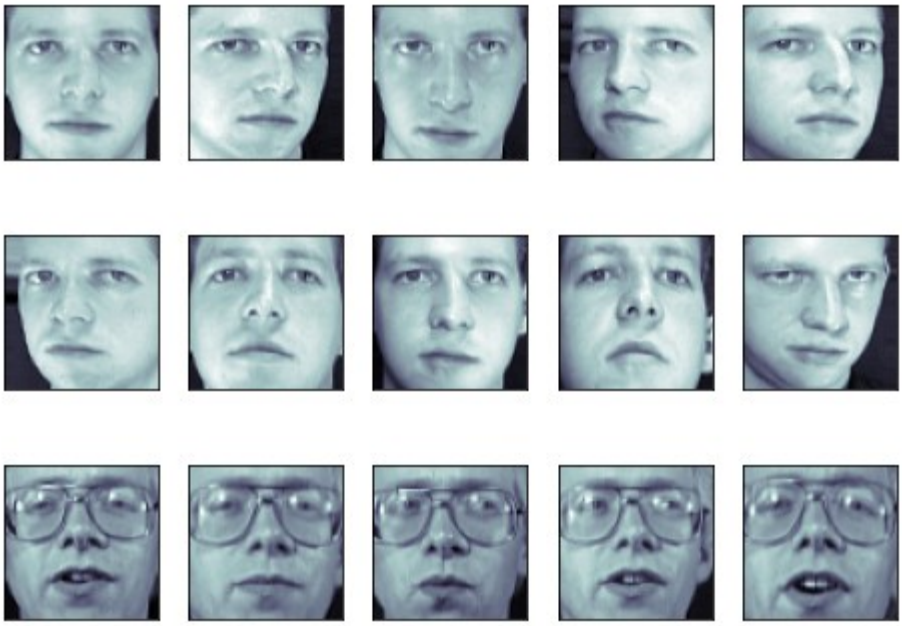
Output

```
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027
(400, 4096)
```

9. Visualisasikan data dengan syntax berikut.

```
from matplotlib import pyplot as plt
fig = plt.figure(figsize=(8, 6))
# plot several images
for i in range(15):
    ax = fig.add_subplot(3, 5, i + 1, xticks=[], yticks=[])
    ax.imshow(faces.images[i], cmap=plt.cm.bone)
```

Output



10. Lakukan train test split pada gambar dengan syntax berikut.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(faces.data,
        faces.target, random_state=0)

print(X_train.shape, X_test.shape)
```

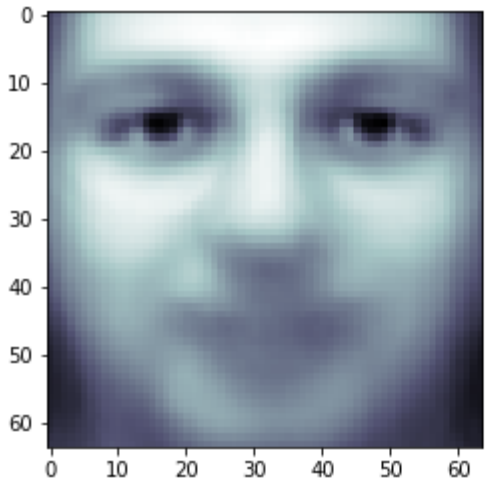
11. Lakukan preprocessing data dengan PCA

```
from sklearn import decomposition
pca = decomposition.PCA(n_components=150, whiten=True)
pca.fit(X_train)
```

```
plt.imshow(pca.mean_.reshape(faces.images[0].shape),
           cmap=plt.cm.bone)
```

Output

```
<matplotlib.image.AxesImage at 0x7fdfc6de9f98>
```




```
print(pca.components_.shape)
```

```
fig = plt.figure(figsize=(16, 6))
for i in range(30):
    ax = fig.add_subplot(3, 10, i + 1, xticks=[], yticks=[])
    ax.imshow(pca.components_[i].reshape(faces.images[0].shape),
              cmap=plt.cm.bone)
```

Output

```
(150, 4096)
```



```
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)
print(X_train_pca.shape)
```

```
print(X_test_pca.shape)
```

Output

```
(300, 150)

(100, 150)
```

REFERENSI

4. docs.opencv.org
5. scipy-lectures.org
6. medium.com

MODUL 9
CNN UNTUK KLASIFIKASI GAMBAR (1)

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mengimplementasikan algoritma CNN sederhana untuk klasifikasi gambar.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menguji metode neural networks dalam analisis gambar dengan Pyhton (C4)

PENUNJANG PRAKTIKUM

1. Jupyter Lab/Google Colab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Pada modul ini akan diimplementasikan klasifikasi gambar sederhana menggunakan MNIST dataset dan Keras. Klasifikasi gambar dapat diimplementasikan dengan beberapa step utama

- Persiapan data
- Melakukan pra-pemrosesan data dan augmentasi data (jika diperlukan)
- Membagi data menjadi *train* data dan *test* data
- *Build model*
- *Compile model*
- Latih model

Catatan : Syntax pada modul ini dijalankan pada Google Colab.

Analisis Dataset

1. Dataset yang digunakan merupakan MNIST dataset pada library Keras.

```
#import library
import tensorflow as tf
from tensorflow import keras
```

2. Import dataset dengan syntax berikut.

```
mnist = tf.keras.datasets.fashion_mnist
```

MNIST dataset merupakan dataset yang terdiri dari 70.000 gambar *fashion clothing* berwarna *grayscale* dan beresolusi rendah dengan 10 kelas (kategori). Resolusi pada setiap gambar adalah 28x28 pixel.

3. Pisahkan data latih dan data test

```
#split data
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

Syntax di atas digunakan untuk membagi dataset menjadi 60.000 data latih da 10.000 data test

4. Lakukan normalisasi data

```
#basic preprocess (normalize data)
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Sebelum dilatih menggunakan *neural networks*, dataset perlu diubah *value* nya sehingga berada di range antara 0 dan 1. Sehingga perlu dilakukan normalisasi data menggunakan syntax di atas.

5. *Build model*

```
#build model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
```

Bagian dasar dari *neural network* adalah layer. *Layer* berfungsi untuk mengekstrak informasi yang ada pada data. Secara umum, layer terdiri dari 3 bagian utama

- Input layer : input layer merupakan layer yang memiliki parameter input shape, yaitu parameter yang memberikan informasi mengenai ukuran pixel dari gambar data train. Dalam contoh ini, ukuran gambar adalah 28x28 pixel sehingga input shapenya adalah (28,28)
  Layer flatten berfungsi untuk ,meratakan input, yaitu mengubah gambar yang awalnya 2 dimensi (28 x 28) menjadi array 1 dimensi. (784)
- Hidden layer : dense layer pada Keras berfungsi sebagai hidden layer pada *neural networks (*bisa digunakan juga sebagai output layer). Pada layer ini juga perlu dimasukkan activation layer (akan dibahas lebih detail di akhir modul)
- Output layer : pada output layer, jumlah unit yang digunakan menyesuaikan jumlah label pada dataset. Dalam contoh ini, digunakan nilai 10 karena pada mnist dataset terdapat 10 kelas. Pada output layer juga bisa ditambahkan fungsi aktivasi.

6. *Compile model*

```
#compile model
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

Sebelum data dilatih dengan model, perlu dilakukan *compile model.* Pada bagian ini, ada beberapa seeting yang perlu ditambahkan yaitu

- Loss function : digunakan untuk mengukur seberapa akurat model selama training
- Optimizer : digunakan untuk menyesuaikan bagaimana model diupdate berdasarkan data yang sudah dilihat dan loss function nya
- Metrics – digunakan untuk memonitor selama step training dan testing. Dalam contoh di atas digunakan accuracy, yaitu metric yang digunakan untuk mengukur berapa presentse gambar yang berhasil diklasifikasikan.

7. Latih model

```
#train the model to the mnist dataset
model.fit(train_images, train_labels, epochs=10)
```

Hasil latih model ditunjukkan pada gambar terlampir

```
Epoch 1/10
1875/1875 [==============================] - 5s 2ms/step - loss: 0.4974 - accuracy: 0.8243
Epoch 2/10
1875/1875 [==============================] - 5s 2ms/step - loss: 0.3760 - accuracy: 0.8646
Epoch 3/10
1875/1875 [==============================] - 4s 2ms/step - loss: 0.3367 - accuracy: 0.8777
Epoch 4/10
1875/1875 [==============================] - 5s 3ms/step - loss: 0.3122 - accuracy: 0.8859
Epoch 5/10
1875/1875 [==============================] - 4s 2ms/step - loss: 0.2934 - accuracy: 0.8912
Epoch 6/10
1875/1875 [==============================] - 5s 2ms/step - loss: 0.2805 - accuracy: 0.8968
Epoch 7/10
1875/1875 [==============================] - 5s 2ms/step - loss: 0.2679 - accuracy: 0.9005
Epoch 8/10
1875/1875 [==============================] - 5s 3ms/step - loss: 0.2572 - accuracy: 0.9049
Epoch 9/10
1875/1875 [==============================] - 5s 3ms/step - loss: 0.2489 - accuracy: 0.9060
Epoch 10/10
1875/1875 [==============================] - 5s 2ms/step - loss: 0.2398 - accuracy: 0.9109
<keras.callbacks.History at 0x7fdce9e73e50>
```

Training data dapat dilakukan menggunakan syntax “fit”. Hasil pada gambar di atas menunjukkan bahwa model memperoleh akurasi sebesar 91% pads data training.

8. Evaluasi model

```
#model evaluation
test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
print('\nTest accuracy:', test_acc)
```

```
313/313 - 1s - loss: 0.3477 - accuracy: 0.8782 - 551ms/epoch - 2ms/step

Test accuracy: 0.8781999945640564
```

Dari evaluasi model di atas, menunjukkan bahwa model memperoleh akurasi pada test data sebesar 89%

REFERENSI

1. Tensorflow.org

MODUL 10
CNN UNTUK KLASIFIKASI GAMBAR (2)

DESKRIPSI TEMA

Pada pertemuan ini mahasiswa akan mengimplementasikan algoritma CNN sederhana untuk klasifikasi gambar.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menguji metode neural networks dalam analisis gambar dengan Pyhton (C4)

PENUNJANG PRAKTIKUM

1. Jupyter Lab/Google Colab
2. Koneksi internet
3. Aplikasi *browser* (Google Chrome, Mozilla Firefox, dsb.)

LANGKAH-LANGKAH PRAKTIKUM

Pada modul ini kita akan menggunakan dataset mnist dan Library Keras untuk mengimplementasikan klasifikasi gambar sederhana

Persiapan Dataset

1. Dataset yang digunakan dalam praktikum ini berbentuk .zip dengan 2000 gambar kucing dan anjing bertipe JPG.

Import dataset dengan syntax berikut.

```
!wget --no-check-certificate \
    https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip \
    -O /tmp/cats_and_dogs_filtered.zip
```

```
import os
import zipfile

local_zip = '/tmp/cats_and_dogs_filtered.zip'
zip_ref = zipfile.ZipFile(local_zip, 'r')
zip_ref.extractall('/tmp')
zip_ref.close()
```

2. Tentukan directory untuk dataset tersebut.

```
base_dir = '/tmp/cats_and_dogs_filtered'
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')

# Directory with our training cat pictures
train_cats_dir = os.path.join(train_dir, 'cats')

# Directory with our training dog pictures
train_dogs_dir = os.path.join(train_dir, 'dogs')

# Directory with our validation cat pictures
validation_cats_dir = os.path.join(validation_dir, 'cats')

# Directory with our validation dog pictures
validation_dogs_dir = os.path.join(validation_dir, 'dogs')
```

3. Kita dapat melihat penamaan file pada directory "validation" dan "train" menggunakan syntax berikut.

```
train_cat_fnames = os.listdir(train_cats_dir)
print(train_cat_fnames[:10])

train_dog_fnames = os.listdir(train_dogs_dir)
train_dog_fnames.sort()
print(train_dog_fnames[:10])
```

Output

```
['cat.532.jpg', 'cat.362.jpg', 'cat.815.jpg', 'cat.24.jpg', 'cat.478.jpg', 'cat.821.jpg', 'cat.418.jpg', 'cat.234.jpg', 'cat.809.jpg', 'cat.948.jpg']
['dog.0.jpg', 'dog.1.jpg', 'dog.10.jpg', 'dog.100.jpg', 'dog.101.jpg', 'dog.102.jpg', 'dog.103.jpg', 'dog.104.jpg', 'dog.105.jpg', 'dog.106.jpg']
```

4. Untuk melihat jumlah file pada masing-masing directory, gunakan syntax berikut.

```
print('total training cat images:', len(os.listdir(train_cats_dir)))
print('total training dog images:', len(os.listdir(train_dogs_dir)))
print('total validation cat images:', len(os.listdir(validation_cats_dir)))
print('total validation dog images:', len(os.listdir(validation_dogs_dir)))
```

Output

```
total training cat images: 1000
total training dog images: 1000
total validation cat images: 500
total validation dog images: 500
```

5. Untuk menampilkan beberapa contoh gambar dari dataset, gunakan syntax berikut.

```
%matplotlib inline

import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Parameters for our graph; we'll output images in a 4x4 configuration
nrows = 4
ncols = 4

# Index for iterating over images
pic_index = 0
```

```
# Set up matplotlib fig, and size it to fit 4x4 pics
fig = plt.gcf()
fig.set_size_inches(ncols * 4, nrows * 4)

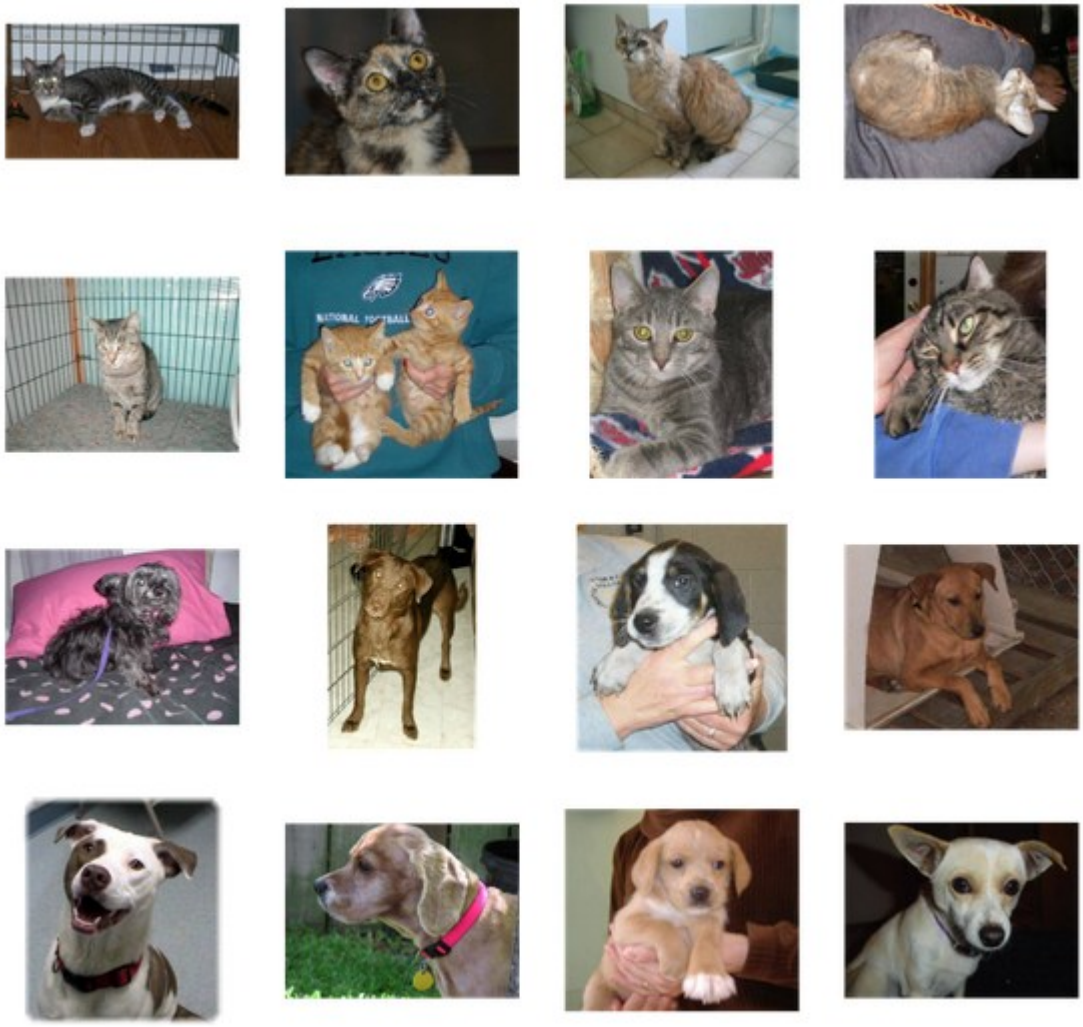
pic_index += 8
next_cat_pix = [os.path.join(train_cats_dir, fname)
                for fname in train_cat_fnames[pic_index-8:pic_index]]
next_dog_pix = [os.path.join(train_dogs_dir, fname)
                for fname in train_dog_fnames[pic_index-8:pic_index]]

for i, img_path in enumerate(next_cat_pix+next_dog_pix):
  # Set up subplot; subplot indices start at 1
  sp = plt.subplot(nrows, ncols, i + 1)
  sp.axis('Off') # Don't show axes (or gridlines)

  img = mpimg.imread(img_path)
  plt.imshow(img)

plt.show()
```

Output



Membuat model CNN sederhana untuk klasifikasi gambar

Gambar yang digunakan dalam training ini berukuran 150x50. Training ini akan menggunakan 3 modul {convolution + relu + maxpooling}. Layer convolution akan beroperasi pada windows 3x3 sedangkan layer maxpooling akan beroperasi pada windows 2x2. Konvolusi pertama akan mengekstrak 16 filter, konvolusi kedua akan mengekstrak 32 filter, sedangkan konvolusi ketiga akan mengekstrak 64 filter.

6. Buat layer model dengan syntax berikut.

```
from tensorflow.keras import layers
from tensorflow.keras import Model
```

```
# Our input feature map is 150x150x3: 150x150 for the image pixels, and 3 for
# the three color channels: R, G, and B
img_input = layers.Input(shape=(150, 150, 3))

# First convolution extracts 16 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(16, 3, activation='relu')(img_input)
x = layers.MaxPooling2D(2)(x)

# Second convolution extracts 32 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(32, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Third convolution extracts 64 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(64, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)
```

7. Karena pada model ini kita menggunakan two-class classification, aktivasi output yang digunakan adalah sigmoid activation.. Untuk multiclass classification, fungsi aktivasi yang digunakan adalah fungsi aktivasi softmax

```
# Flatten feature map to a 1-dim tensor so we can add fully connected layers
x = layers.Flatten()(x)

# Create a fully connected layer with ReLU activation and 512 hidden units
x = layers.Dense(512, activation='relu')(x)

# Create output layer with a single node and sigmoid activation
output = layers.Dense(1, activation='sigmoid')(x)

# Create model:
# input = input feature map
# output = input feature map + stacked convolution/maxpooling layers + fully
# connected layer + sigmoid output layer
model = Model(img_input, output)
```

8. Untuk melihat summay model yang digunakan, tambahkan syntax berikut.

```
model.summary()
```

Output

```
Model: "model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         [(None, 150, 150, 3)]     0
_________________________________________________________________
conv2d (Conv2D)              (None, 148, 148, 16)      448
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 74, 74, 16)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 72, 72, 32)        4640
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 36, 36, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 34, 34, 64)        18496
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 17, 17, 64)        0
_________________________________________________________________
flatten (Flatten)            (None, 18496)             0
_________________________________________________________________
dense (Dense)                (None, 512)               9470464
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 513
=================================================================
Total params: 9,494,561
Trainable params: 9,494,561
Non-trainable params: 0
```

Compile Model

9. Model ini akan ditraining menggunakan binary_crossentropy loss (karena diimplementasikan untuk klasifikasi two-class dan aktivasi outputnya menggunakan sigmoid function). Optimizer yang digunakan pada training ini adalah RMSProp optimizer dengan learning rate 0.001. Optimizer lain seperti Adam dan Adagrad optimizer dapat juga digunakan untuk training model ini.

```
from tensorflow.keras.optimizers import RMSprop

model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(learning_rate=0.001),
              metrics=['acc'])
```

Data Preprocessing

10. Dataset pada neural networks harus dinormalisasi sehingga pada contoh ini, pixel pada setiap gambar akan berada diantara nilai [0,1]

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# All images will be rescaled by 1./255
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)

# Flow training images in batches of 20 using train_datagen generator
train_generator = train_datagen.flow_from_directory(
        train_dir,  # This is the source directory for training images
        target_size=(150, 150),  # All images will be resized to 150x150
        batch_size=20,
        # Since we use binary_crossentropy loss, we need binary labels
        class_mode='binary')

# Flow validation images in batches of 20 using val_datagen generator
validation_generator = val_datagen.flow_from_directory(
        validation_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode='binary')
```

Output

```
Found 2000 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
```

Training Dataset

11. Lakukan training dataset menggunakan 15 epoch lakukan validasi untuk 1000 gambar

```
history = model.fit(
      train_generator,
      epochs=15,
      validation_data=validation_generator,
      verbose=2
)
```

Output

```
Epoch 1/15
100/100 - 19s - 186ms/step - acc: 0.9365 - loss: 0.1615 - val_acc: 0.7400 - val_loss: 0.8177
Epoch 2/15
100/100 - 20s - 197ms/step - acc: 0.9675 - loss: 0.1033 - val_acc: 0.7480 - val_loss: 0.8982
Epoch 3/15
100/100 - 19s - 195ms/step - acc: 0.9795 - loss: 0.0560 - val_acc: 0.7170 - val_loss: 1.0554
Epoch 4/15
100/100 - 21s - 210ms/step - acc: 0.9860 - loss: 0.0485 - val_acc: 0.7290 - val_loss: 1.1176
Epoch 5/15
100/100 - 20s - 198ms/step - acc: 0.9865 - loss: 0.0574 - val_acc: 0.7330 - val_loss: 1.2082
Epoch 6/15
100/100 - 18s - 179ms/step - acc: 0.9910 - loss: 0.0364 - val_acc: 0.6860 - val_loss: 1.8330
Epoch 7/15
100/100 - 18s - 176ms/step - acc: 0.9905 - loss: 0.0626 - val_acc: 0.7460 - val_loss: 1.2352
Epoch 8/15
100/100 - 18s - 177ms/step - acc: 0.9885 - loss: 0.0625 - val_acc: 0.7130 - val_loss: 1.2854
Epoch 9/15
100/100 - 18s - 178ms/step - acc: 0.9870 - loss: 0.1289 - val_acc: 0.7430 - val_loss: 1.4456
Epoch 10/15
100/100 - 18s - 184ms/step - acc: 0.9935 - loss: 0.0222 - val_acc: 0.7240 - val_loss: 1.8086
Epoch 11/15
100/100 - 20s - 196ms/step - acc: 0.9990 - loss: 0.0043 - val_acc: 0.7360 - val_loss: 1.9250
Epoch 12/15
100/100 - 19s - 192ms/step - acc: 0.9850 - loss: 0.1180 - val_acc: 0.7370 - val_loss: 1.5719
Epoch 13/15
100/100 - 21s - 212ms/step - acc: 0.9950 - loss: 0.0150 - val_acc: 0.7320 - val_loss: 1.9184
Epoch 14/15
100/100 - 20s - 195ms/step - acc: 0.9960 - loss: 0.0139 - val_acc: 0.7280 - val_loss: 1.7588
Epoch 15/15
100/100 - 19s - 188ms/step - acc: 0.9965 - loss: 0.0135 - val_acc: 0.7340 - val_loss: 1.7540
```

Visualisasi Proses pada Model (Optional)

12. Untuk memvisualisasi proses yang ada di dalam CNN, dapat digunakan syntax berikut.

```
import numpy as np
import random
from tensorflow.keras.preprocessing.image import img_to_array, load_img

# Let's define a new Model that will take an image as input, and will output
# intermediate representations for all layers in the previous model after
# the first.
successive_outputs = [layer.output for layer in model.layers[1:]]
visualization_model = Model(img_input, successive_outputs)

# Let's prepare a random input image of a cat or dog from the training set.
cat_img_files = [os.path.join(train_cats_dir, f) for f in train_cat_fnames]
dog_img_files = [os.path.join(train_dogs_dir, f) for f in train_dog_fnames]
img_path = random.choice(cat_img_files + dog_img_files)
```

```
img = load_img(img_path, target_size=(150, 150))  # this is a PIL image
x = img_to_array(img)  # Numpy array with shape (150, 150, 3)
x = x.reshape((1,) + x.shape)  # Numpy array with shape (1, 150, 150, 3)

# Rescale by 1/255
x /= 255

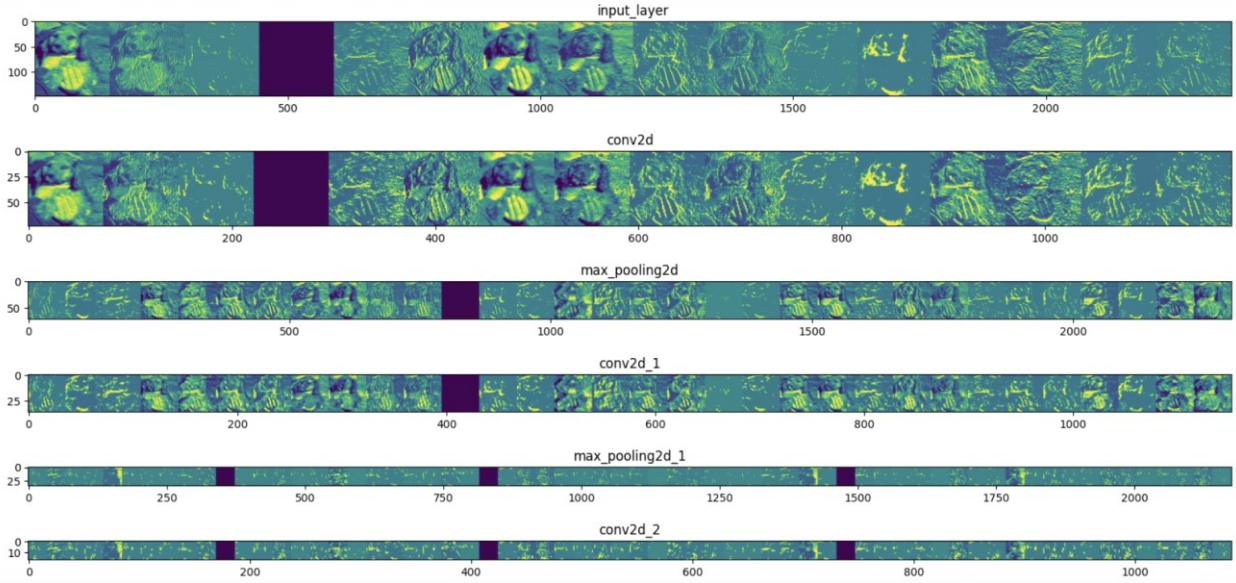
# Let's run our image through our network, thus obtaining all
# intermediate representations for this image.
successive_feature_maps = visualization_model.predict(x)
```

```
# These are the names of the layers, so can have them as part of our plot
layer_names = [layer.name for layer in model.layers]

# Now let's display our representations
for layer_name, feature_map in zip(layer_names, successive_feature_maps):
  if len(feature_map.shape) == 4:
    # Just do this for the conv / maxpool layers, not the fully-connected layers
    n_features = feature_map.shape[-1]  # number of features in feature map
    # The feature map has shape (1, size, size, n_features)
    size = feature_map.shape[1]
    # We will tile our images in this matrix
    display_grid = np.zeros((size, size * n_features))
    for i in range(n_features):
      # Postprocess the feature to make it visually palatable
      x = feature_map[0, :, :, i]
      x -= x.mean()
      x /= x.std()
      x *= 64
      x += 128
      x = np.clip(x, 0, 255).astype('uint8')
      # We'll tile each filter into this big horizontal grid
      display_grid[:, i * size : (i + 1) * size] = x
    # Display the grid
    scale = 20. / n_features
    plt.figure(figsize=(scale * n_features, scale))
    plt.title(layer_name)
    plt.grid(False)
    plt.imshow(display_grid, aspect='auto', cmap='viridis')
```

Output

```
/var/folders/kb/4fw_34mj3jb8ddv55q21lht80000gp/T/ipykernel_72116/3266021532.py:13: RuntimeWarning: invalid value encoun
tered in divide
  x /= x.std()
/var/folders/kb/4fw_34mj3jb8ddv55q21lht80000gp/T/ipykernel_72116/3266021532.py:16: RuntimeWarning: invalid value encoun
tered in cast
  x = np.clip(x, 0, 255).astype('uint8')
```



Evaluasi hasil training dan validasi akurasi

13. Gunakan syntax berikut untuk melihat hasil plot training dan validasi akurasi dari model yang telah kita implementasikan.

```
# Retrieve a list of accuracy results on training and validation data
# sets for each training epoch
acc = history.history['acc']
val_acc = history.history['val_acc']

# Retrieve a list of list results on training and validation data
# sets for each training epoch
loss = history.history['loss']
val_loss = history.history['val_loss']

# Get number of epochs
epochs = range(len(acc))

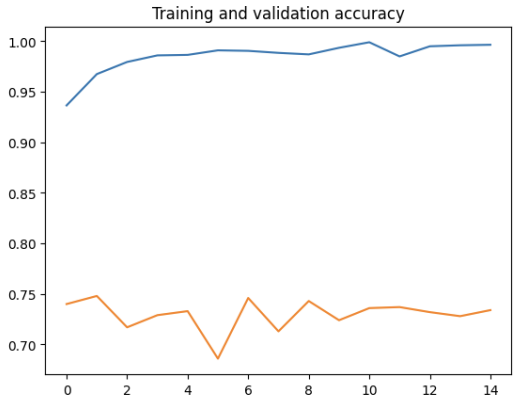
# Plot training and validation accuracy per epoch
plt.plot(epochs, acc)
plt.plot(epochs, val_acc)
plt.title('Training and validation accuracy')

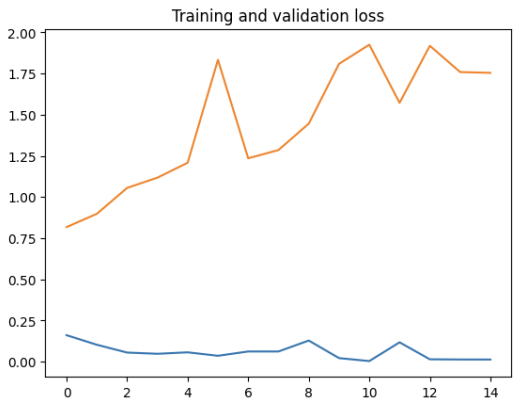
plt.figure()

# Plot training and validation loss per epoch
plt.plot(epochs, loss)
plt.plot(epochs, val_loss)
plt.title('Training and validation loss')
```

Output

```
Text(0.5, 1.0, 'Training and validation loss')
```





REFERENSI

1. docs.opencv.org
2. scipy-lectures.org
3. medium.com